Electronics and Computer Science
Faculty of Engineering and Physical Sciences
University of Southampton

Nabil Al-Shanteer

29th of April, 2025

Deep Learning for Activity Recognition with Application to Surveillance

Project supervisor: Zhiwu Huang
Second examiner: Jize Yan

A project report submitted for the award of
MEng Electronic Engineering with Artificial Intelligence

Abstract

Canonical surveillance action recognition pipelines traditionally rely on modality-specific models using handcrafted features or supervised CNN architectures, limiting their adaptability to evolving behaviours in low-resource training. This project advances the investigation of **ActionCLIP**, a vision-language model, for surveillance contexts under low-resource CPU constraints. The first phase successfully reproduces few-shot learning on the UCF-101 dataset, achieving top-1 accuracies of 60%, 62.19%, and 63.61% for 1-shot, 2-shot, and 4-shot settings respectively. The second phase adapts **ActionCLIP** to the UCF Crime dataset, initially employing binary classification between the classes *NormalVideos* and *Shoplifting*; however, conventional fine-tuning yielded a low 28.18% accuracy. To dissect this limitation, the textual encoder was enhanced via prompt engineering, improving performance to 44.13% without additional training. Additionally, reframing the problem as an unsupervised anomaly detection task using Mahalanobis distance over visual embeddings achieved 97.56% accuracy and a ROC-AUC of 0.9913. The findings of this study reveal that **ActionCLIP**'s visual backbone, ViT-B/16, models rich spatiotemporal features, while its text encoder remains highly sensitive to linguistic design. The project concludes by demonstrating the model's potential for surveillance deployment, while proposing future refinements in lightweight, resource-constrained vision-language models.

Statement of Originality

- I have read and understood the ECS Academic Integrity information and the University's Academic Integrity Guidance for Students.
- I am aware that failure to act in accordance with the Regulations Governing Academic Integrity may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

You must change the statements in the boxes if you do not agree with them.

We expect you to acknowledge all sources of information (e.g. ideas, algorithms, data) using citations. You must also put quotation marks around any sections of text that you have copied without paraphrasing. If any figures or tables have been taken or modified from another source, you must explain this in the caption and cite the original source.

I have acknowledged all sources, and identified any content taken from elsewhere.

If you have used any code (e.g. open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

I have used resources produced by others such as the ActionCLIP GitHub codebase, UCF-Crime and UCF-101 datasets. MNIST was used for the Progress Report content.

You can consult with module teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly-accessible repositories e.g. Github, unless expressly permitted by the module leader), or help them to do theirs. For individual assignments, we expect you to work on your own. For group assignments, we expect that you work only with your allocated group. You must get permission in writing from the module teaching staff before you seek outside assistance, e.g. a proofreading service, and declare it here.

I did all the work myself, or with my allocated group, and have not helped anyone else.

We expect that you have not fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (either in the report, or submitted as a separate file) so that your results could be reproduced.

The material in the report is genuine, and I have included all my data/code/designs.

We expect that you have not previously submitted any part of this work for another assessment. You must get permission in writing from the module teaching staff before re-using any of your previously submitted work for this assessment.

I have not submitted any part of this work for another assessment.

If your work involved research/studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must

give details of this in the report, and list the ethical approval reference number(s) in the box below.

My work did not involve human participants, their cells or data, or animals.

ECS Statement of Originality Template, updated August 2018, Alex Weddell aiofficer@ecs.soton.ac.uk

Acknowledgements

I would like to first and foremost thank my project supervisor, Dr. Zhiwu Huang, for their continuous support and regular checkups throughout the timeline of this project. Weekly meetings were not mandatory by the University, but were a privilege that Dr. Huang personally provided for guidance and advice. His resilience, patience, and academic feedback were greatly appreciated. Most notably, I am thankful for the access of resources given to me for the Computer Vision and Deep Learning modules in which as a beginner to machine learning, I learned a lot.

I am also grateful to the University of Southampton's iSolutions team for their assistance during my attempts to access the Iridis 5 HPC cluster. Though the original plan to utilise GPU-based training was not realised, their correspondence helped clarify important constraints early on.

I gratefully acknowledge the authors of the **ActionCLIP** open-source repository, whose codebase served as the foundation for this work. I would also like to thank the developers and maintainers of the UCF-101 and UCF Crime datasets, which made this investigation possible.

Contents

Abstract ii
Statement of Originality iii
Acknowledgements v
Contents vi
1 Introduction 1
2 Background Reading 3
2.1 Evolution of Action Recognition Models 3
2.1.1 HOF and iDT 3
2.1.2 2D CNNs 4
2.1.3 Two-Stream CNNs 4
2.1.4 3D CNNs 5
2.1.5 SlowFast Networks 5
2.1.6 TimeSformer 6
2.1.7 ActionCLIP 6
2.2 Architectural Dissection of ActionCLIP 7
2.2.1 Frame Sampling 7
2.2.2 Pre-Network Prompt 8
2.2.3 In-Network Prompt 9
2.2.4 Post-Network Prompt 11
2.2.5 Label Pipeline and Classification 12
3 Project Background 14
4 Experimental Design 15
4.1 Dataset Selection 15
4.2 Implementation of ActionCLIP: Methodology 18
4.2.1 Fine-Tuning Method 18
4.2.2 Prompt Engineering Method 19
4.2.3 Anomaly Detection Method 21
5 Results and Evaluation 22
5.1 Few-Shot Learning on UCF-101 22
5.2 Fine-Tuning on UCF Crime 23
5.2.1 Prompt Engineering Approach 23
5.2.2 Anomaly Detection Approach 25
6 Conclusions 27
References 29
Appendix A. Informative Notes 34
A.1 Frame Sampling 34
A.2 Pre-Network Prompt 35
A.3 In-Network Prompt 36
A.4 Post-Network Prompt 37
A.5 Preamble Notes 39

Appendix B. Selected Listings 49
B.1 environment.yml (Modified) 49
B.2 frame_extract.py 50
B.3 validation_split.py 51
B.4 generate_crime.py 51
B.5 train.py (Modified) 52
B.6 test.py (Modified) 56
B.7 ucf_train_1shot.yaml 60
B.8 ucf_train_2shot.yaml 60
B.9 ucf_train_4shot.yaml 61
B.10 ucf_crime_train.yaml 62
B.11 prompt_ensemble.py 63
B.12 anomaly_test.py 64
Appendix C. Project Planning and Risk Assessment 68
C.1 Proposed Project Timeline: Gantt Chart of Initial Plan 68
C.2 Actual Project Timeline: Gantt Chart of Final Execution 68
C.3 Risk Assessment Form 69
C.4 Project Brief 70
Appendix D. ERGO II Secondary Data Analysis Ethics Approval 71
Appendix E. Archive Folder Contents 72

1 Introduction

The proliferation of intelligent surveillance technology in contemporary society underscores the urgency to develop autonomous systems capable of recognising human activities. The United Nations, in their recent paper, *Global Study on Homicide 2023*, proclaimed that the urbanisation of the global population will increase from 54 per cent to 68 per cent by 2050 and have identified that *"the underlying conditions in fast-growing cities… are the criminogenic factors"* [1]. Urban environments worldwide are expanding at unprecedented rates and are accompanied by a surge in day-to-day security threats that demand vigilant real-time monitoring. Surveillance infrastructure now serves as the silent guardian of public safety, yet its efficacy remains tied to the limitations of its analytical frameworks – traditional *Human Action Recognition* (HAR) systems, which often rely on hand-engineered features and fixed classifiers. While effective in controlled scenarios, they lack the versatility to distinguish between benign behaviours and harmful actions when presented with novel or ambiguous situations.

Despite advancements in deep learning architectures, HAR within surveillance footage remains a pragmatic challenge due to the nature of video data. Far from static imagery, video sequences are multidimensional – embedding spatial features like objects' arrangement as well as temporal features which encapsulate the flow of motion across frames. Compounding this data structure are variables that are common in surveillance contexts: (i) unsteady lighting conditions, (ii) fluctuating frame rates, (iii) frequent occlusions, and (iv) fluctuations in video resolution – all conspiring to complicate action recognition.

These factors contribute to HAR model performance degradation when transitioning from controlled, curated datasets to unpredictable real environments. For example, subtle gestures associated with shoplifting may evade recognition by conventional models due to their struggle to discern actions when visual cues are obscured or presented in unfamiliar contexts. Our model's elevated performance in anomaly detection tasks indicates that **ActionCLIP**'s latent representations are more invariant to these types of noise typically encountered in surveillance footage, achieving a 99.13% separability score between anomalous and benign behaviour.

Traditional HAR frameworks depend on extensive labelled datasets for supervised learning, thus necessitating comprehensive retraining whenever new behavioural patterns emerge. However, the evolution of criminal tactics – ranging from simple methods of retail theft to coordinated acts of public disorder – outpaces the adaptability of these models, leaving them under-equipped to recognise actions beyond their training scope.

Early approaches to action recognition predominantly relied on manually engineered features, such as *Histograms of Oriented Gradients* (HOG) and *Histograms of Optical Flow* (HOF), paired with conventional classifiers like *Support Vector Machines* (SVMs). While they provided the foundation for classifying human movement, they depended heavily on accurately annotated datasets and lacked flexibility in newfound environments. The advent of deep learning, in particular the advancement of *Convolutional Neural Networks* (CNNs), helped bring significant progress by supporting automatic extraction of spatial and temporal patterns from raw video data. Even so, those models required large amounts of labelled footage, as well as iterative retraining to adapt to new action classes.

This reliance on exhaustive data curation and retraining presents a bottleneck when confronting emergent unlabelled behaviours. Surveillance environments are characterised by unpredictability; criminals continuously devise novel tactics that evade recognition – whether conducted by human security or rigidly trained models. Bridging this gap demands a paradigm shift towards models capable of recognising unseen actions based on semantic understanding without requiring prior exposure. The proposed solution is *zero-shot* learning, which refers to the leverage of semantic relationships to infer knowledge about new classes – the basis for multimodal architectures that unify visual and textual modalities for flexible action recognition.

This project addresses the highlighted limitations by investigating the adaptability of **ActionCLIP**, a state-of-the-art multimodal framework originally designed for video action recognition, which will be used for deployment in surveillance contexts. **ActionCLIP** redefines HAR by aligning video inputs with rich, descriptive textual prompts, acquiring the potential for high performing zero-shot classification. Through this research, the aim is to harness **ActionCLIP's** capacity for semantic generalisation to detect criminal and suspicious activities without reliance on exhaustive retraining, thereby enhancing the adaptability of surveillance systems in low-resourced dynamic environments.

2 Background Reading

The field of HAR has undergone significant transformations, evolving from handcrafted feature-based methods to sophisticated deep learning architectures. This progression reflects the continuous pursuit of models that can accurately and efficiently interpret human activities from visual data.

2.1 Evolution of Action Recognition Models

Accurately recognising human actions in video data requires solving for two fundamental factors: *spatial structure* – describing the arrangement of visual features within frames, and *temporal dynamics* – the progression of these features over time. Formally, let $X = \{x_t\}_{t=1}^{T}$ represent a sequence of video frames over time T. The task is to learn a function f that maps $X \longrightarrow Y$, where Y is the action class label space.

2.1.1 HOF and iDT

Initial methods manually engineered feature representations Φ from video sequences by tracking local pixel displacement to reproduce motion information. Techniques such as HOF and *Improved Dense Trajectories* (iDT) extracted dense motion descriptors by approximating the optical flow field $F = (u, v)$, where u and v denote the horizontal and vertical components of motion [2][3]. Under the *brightness constancy assumption*, Equation (1) arises [4].

$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t) \tag{1}$$

Then by linearising, the optical flow constraint Equation (2) arises [4].

$$\frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v + \frac{\partial I}{\partial t} = 0 \tag{2}$$

Further quantising these descriptors into histograms creates static vector representations $\Phi \in \mathbb{R}^d$, which were inputted to shallow classifiers such as SVMs. Illustrated in Figure 1 is the dense sampling and trajectory description pipeline employed in iDT.

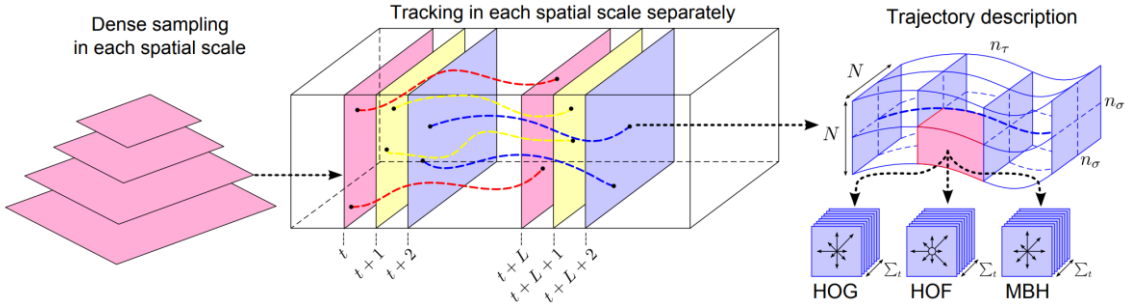


Figure 1: Visualisation of iDT pipeline taken from [9], combining dense trajectory tracking with HOG, HOF, and MBH descriptors to generate quantised histogram-based feature representations $\Phi \in \mathbb{R}^d$ for action classification.

However, these methods suffered from static feature parametrisation – the mapping of $X \longrightarrow \Phi$ was fixed by design, hard-coded using mathematical rules and processing steps, and could not adaptively adjust to variations in illumination, occlusion, or background clutter, leading to poor generalisation in complex scenes [5][7]. This is because optical flow approximates how pixels move across time by assuming the brightness constancy [6] – that the appearance of a pixel stays the same as it moves. Modern deep models do not

rely on this assumption, primarily because HOF and iDT fail when the brightness constancy assumption does not hold, rather, they train end-to-end to learn representations from raw pixel data across time which proves to be more efficient [10].These recent models have been said to have *"surpassed traditional energy-optimization-based methods and have been emerging as a major tendency towards improving optical flow estimation."* [8] reflecting the shift towards end-to-end learning frameworks that do not rely on the brightness constancy assumption.

2.1.2 2D CNNs

The rigidity of handcrafted pipelines was overcome by the advent of CNNs which enabled data-driven learning of spatial hierarchies through convolutional kernels [11]. The convolution operation is defined in Equation (3).

$$f_\theta(x) = (x * w) + b \tag{3}$$

Where $w \in \mathbb{R}^{k \times k}$ is the learnable kernel and b is the bias term. Once nonlinear activation functions like *ReLU* $\sigma(x) = \max(0, x)$ emerged, they helped accelerate convergence by avoiding the saturation that was present in the use of *sigmoid* or *tanh* activations, further introducing nonlinearity and allowing CNNs to approximate complex decision boundaries efficiently [12][13].

Pooling layers conferred translation invariance, while end-to-end gradient-based optimisation improved susceptibility to visual distortions [14]. However, 2D CNNs processed frames independently $f_\theta(x_t)$, ignoring temporal dependencies across time, t [15]. As shown in Figure 2, CNN architectures process frames through sequential convolutional and pooling layers, but they treat each frame independently thus disregarding temporal evolution across sequences. Consequently in deep fake studies, they reduced the problem to static frame classification, unable to capture inter-frame motion $\Delta x_t = x_{t+1} - x_t$ [16].

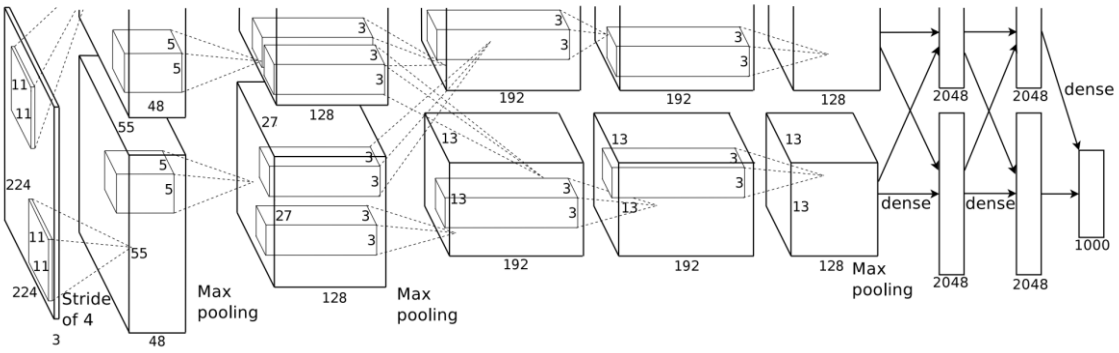


Figure 2: Schematic of a CNN architecture taken from [13], showing independent frame processing through convolutional, pooling, and dense layers.

2.1.3 Two-Stream CNNs

To bridge the temporal gap, *Two-Stream Networks* decomposed action recognition into spatial and temporal streams [17]. The spatial stream consumed RGB frames X_{RGB}, while the temporal stream operated on precomputed optical flow fields $F = (u, v)$[25][23], encoding short-term motion as seen in Equation (4).

$$f_{\theta_{temporal}}(F) = \sigma(W * F + b) \tag{4}$$

This design injected motion information into the learning process, providing the means for distinguishment between visually similar actions with different temporal signatures [24].

Yet, the reliance on external optical flow computation introduced a critical limitation – the flow estimation F was not differentiable with respect to network parameters, breaking end-to-end differentiability [26]. Furthermore, optical flow covered only short temporal horizons $T' \ll T$, failing to capture long-range dependencies inherent in time-extended activities [25] such as *"loitering"* or *"burglary"* surveillance. Computational costs doubled due to dual-stream inference, further impeding scalability [18].

2.1.4 3D CNNs

The *Inflated 3D ConvNet* (I3D) advanced the field by jointly learning spatiotemporal features through volumetric convolutions [19][27]. Extending 2D kernels $w \in \mathbb{R}^{k \times k}$ to 3D $w \in \mathbb{R}^{k \times k \times d}$ [29], I3D captured temporal evolution within convolution layers [28] using the operation described in Equation (5).

$$f_\theta(X) = \sigma\left(\sum_{i,j,t} w_{i,j,t} \cdot X_{i,j,t} + b\right) \tag{5}$$

This architectural enhancement enabled end-to-end temporal feature learning without reliance on precomputed motion cues, as depicted in Figure 3, where 3D convolutions process video sequences holistically to capture spatiotemporal dependencies [30][32].

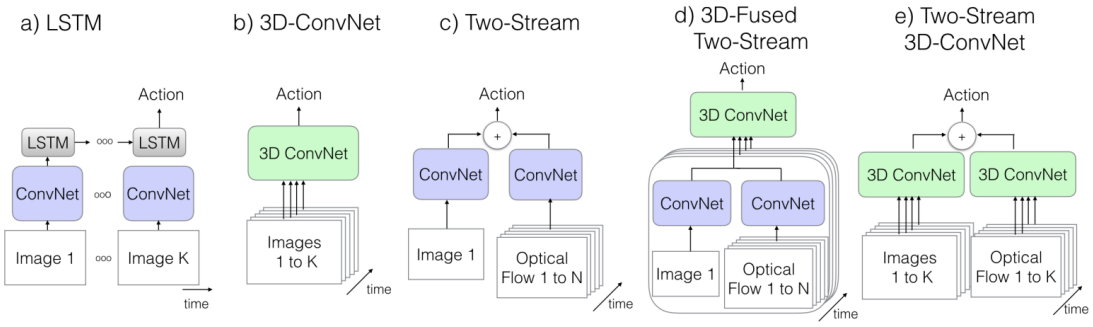


Figure 3: Comparative architectures for action recognition taken from [19], transitioning from frame-level LSTM models and Two-Stream CNNs to spatiotemporal modelling using 3D convolutions.

However, 3D convolutions have increased memory demands due to their cubic computational complexity $O(k^3)$ [34], as well as constraints involving temporal receptive fields to short clips due to GPU limitations [33]. Effective training required massive labelled datasets, *e.g.* ImageNet [35], an impractical expectation for surveillance scenarios with sparse and imbalanced data distributions [19].

2.1.5 SlowFast Networks

Building upon 3D CNNs, *SlowFast Networks* introduced a dual-pathway architecture operating at different temporal resolutions [20]. The *Slow* pathway processed low-frame-rate sequences X_{slow}, focusing on semantic content, while the *Fast* pathway consumed high-frame-rate inputs X_{fast}, capturing rapid motion [36]. The result of both pathways are concatenated as shown in Equation (6).

$$f(X) = f_s(X_{slow}) + f_f(X_{fast}) \tag{6}$$

This design improved sensitivity to both slow and fast motion patterns, making it effective for distinguishing fine-grained actions [20]. Nonetheless, SlowFast networks increased the computational cost [37] and remained dependent on large-scale supervised datasets for training [38]. While they expanded temporal coverage, they still lacked semantic understanding of the underlying action context – models recognised motion patterns but not their semantic implications [25][36], *e.g.* distinguishing "running for exercise" from "running to evade security".

2.1.6 TimeSformer

Transformers introduced global context aggregation through self-attention mechanisms [21], enabling the model to compute dependencies between any two frames regardless of their distance in time [39] as depicted by Equation (7).

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{7}$$

The *TimeSformer* leveraged factorised attention over spatial and temporal dimensions, significantly improving long-range dependency modelling [21]. While TimeSformer mitigated convolutional locality constraints, it inherited the data-hungry nature of transformers, requiring extensive pretraining to stabilise learning [41]. Though, there have been experimental attempts at *"alleviating the data-hungry issue of ViT to some extent"* [41] using *Masked Autoencoders* (MAE) by randomly masking large portions of the input data and training the model to reconstruct the missing content.

2.1.7 ActionCLIP

To transcend the limitations of modality-specific architectures, **ActionCLIP** unifies visual and textual modalities into a shared embedding space [22][42]. The model encodes video clips V and textual descriptions T via modality-specific encoders $f_v(V)$ and $f_t(T)$, respectively [43] . Learning is governed by a contrastive loss [44] computed using Equation (8).

$$\mathcal{L} = -\log\frac{\exp\left(\langle f_v(V), f_t(T)\rangle/\tau\right)}{\sum_j \exp\left(\langle f_v(V), f_t(T_j)\rangle/\tau\right)} \tag{8}$$

Where τ is a learnable temperature parameter.

This formulation enables zero-shot action recognition, as seen in Figure 4, allowing the model to generalise to unseen actions through semantic alignment with textual prompts [22]. **ActionCLIP** bridges the semantic gap between visual features and linguistic context, a crucial capability for surveillance scenarios with evolving and unpredictable behaviours.

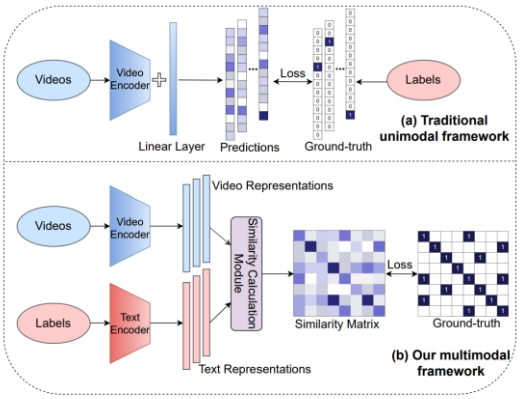


Figure 4: Comparison between traditional unimodal frameworks and the multimodal approach of **ActionCLIP**, taken from [22].

ActionCLIP remains sensitive to prompt engineering and relies on diverse pretraining regimes to capture rare action semantics adequately [22]. Moreover, fine-grained video-text alignment remains an open research challenge [42][43].

2.2 Architectural Dissection of ActionCLIP

ActionCLIP presents a sophisticated extension of *Contrastive Language-Image Pretraining* (CLIP) frameworks that are tailored for video understanding by integrating temporal semantics into the architecture. **ActionCLIP** addresses the dual challenge of encoding spatiotemporal patterns and aligning these with textual prompts for generalisation. This section provides a deep technical dissection of **ActionCLIP**, tracing the entire computational trajectory from raw video frame sampling to classification.

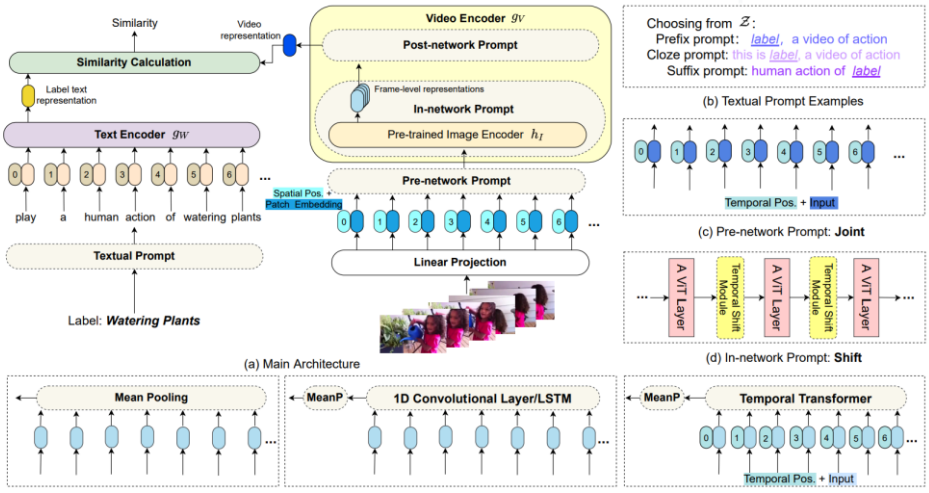


Figure 5: Overview of video and textual pipelines of **ActionCLIP** taken from [22]

Drawing upon the architectural blueprint composed in Figure 5, we follow the progression of data through sampling, embedding projection, multimodal prompting, and cross-modal alignment *(similarity calculation)* to expose the inner workings of this system.

2.2.1 Frame Sampling

The initial step in **ActionCLIP** involves partitioning continuous video streams into discrete frame segments, a preprocessing step that balances temporal coverage and computational feasibility [22]. Instead of processing every frame, the model adopts a

uniform segment-based sampling strategy which we will refer to as *sampling via segmentation* [45][22]. Given an input video V, the total frames are evenly divided into S segments from which a single representative frame f_s is randomly or uniformly selected.

$$V = \{f_s\}_{s=1}^{S}$$

This strategy solidifies the capture of temporal diversity across the video's duration through the sampled frames; the frames selected will guarantee that at least a set of beginning, intermediate, and end frames are taken into account in the classification process, regardless of the video's length. Also aiding in capturing both rapid motions and long duration activities while being in control of input dimensionality [22][46].

Subsequently, each *sampled frame* f_s undergoes spatial decomposition through patch partitioning, a process inspired by *Vision Transformers* (ViT) [41][46]. Each frame gets segmented into a grid of *separate patches* $p_{(x,y)} \in \mathbb{R}^{P\times P\times C}$, where P is the patch size and C is the number of channels (usually in RGB, $C = 3$). Flattening each patch yields a vector in the space $\mathbb{R}^{P^2\cdot C}$ which prepares it for linear projection. These steps can be seen sequentially with an example provided in *Appendix A.1*.

All *flattened patches* are then linearly projected into a high dimensional embedding space using the same learnable weight matrix seen in Equation (9), defined by $E \in \mathbb{R}^{(P^2\cdot C)\times D}$, where D is the model's embedding dimension.

$$z_{(x,y)} = E \cdot p_{(x,y)}^{flat} \tag{9}$$

The result is a sequence of *patch embeddings* which can be portrayed as $\boldsymbol{Z} \in \mathbb{R}^{N\times D}$ [41], where $N = \frac{H\times W}{P^2}$, with H and W representing the frame's height and width respectively.

To retain spatiotemporal ordering, **ActionCLIP** augments these patch embeddings $z_{(x,y)}$ with positional encodings as seen in Equation (10), supplying them with information like spatial position embeddings $e_{(x,y)}^{spatial}$ and frame-level temporal embeddings $e_s^{temporal}$ [21][22]. This step is key to ensuring the transformer can distinguish between identical visual values appearing in different spatial order or timeframes [48].

$$z_{(x,y,s)}^{pos} = z_{(x,y)} + e_{(x,y)}^{spatial} + e_s^{temporal} \tag{10}$$

This positionally encoded sequence forms the input to the subsequent prompt-based augmentation stage [22][49]. At this stage, **ActionCLIP** effectively transforms the raw visual input V into a structurally organised representation via the patch embedding sequence $\boldsymbol{Z}^{pos}$, ready for multimodal interaction.

2.2.2 Pre-Network Prompt

Upon generation of *spatial patch embeddings*, the **ActionCLIP** pipeline supplements these representations with temporal metadata through pre-network prompting [22]. While the embeddings at this stage encapsulate the local visual content of patches and their spatial locations [51][52], they lack awareness of their *temporal order* within the sequence. This omission poses a risk where identical spatial features appearing at different times could be mistakenly interpreted as equivalent by the model. The introduction of *temporal positional encoding* remedies this by imparting a frame-specific signature to each patch embedding [53].

ActionCLIP employs a deterministic, sinusoidal formulation to compute temporal positional encodings, adapted from the foundational transformer architecture, specifically tuned for video sequences [22][54]. For an input sequence of T frames, the positional embedding vector $PE \in \mathbb{R}^D$ for each frame position pos is defined in (11) and (12).

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{D}}}\right) \quad (11), \qquad PE_{pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{D}}}\right) \quad (12)$$

Where $pos \in \{0, 1, \dots, T-1\}$ denotes the temporal index of the frame, i indexes the embedding dimension, and D is the total flattened embedding dimension. This design offers two critical advantages. Since the encoding is not learned but computed algorithmically, the model remains agnostic to the specific length of input sequences, meaning flexible deployment without retraining for varying video lengths [54] – an important property for surveillance applications where clip durations are unpredictable.

Secondly, the alteration between sine and cosine functions introduces a 90° phase shift, making sure that each temporal position generates a unique embedding pattern. This periodicity facilitates the model's ability to infer relative temporal distances between frames through linear operations, without requiring explicit temporal adjacency in the data [55][53]. In other words, even if the model does not explicitly connect neighbouring frames, it can mathematically detect how far apart two frames are by examining their positional embeddings. Notably, this mirrors the natural periodic structures found in temporal data streams, like video [96], and an example of how this is employed can be viewed in *Appendix A.2*.

Importantly, these positional encodings act as *static temporal identifiers*, effectively serving as timestamps for each patch embedding. At this point in the network, there is still no communication between patches from different frames; the embeddings remain isolated, *"without explicit temporal interaction"* [22]. Consequently, the pre-network prompt stage can be interpreted as attaching time-related metadata, informing the model when an event occurs, but not yet how it evolves or relates to surrounding frames [50].

It is worth noting that although **ActionCLIP** employs fixed sinusoidal positional encodings, alternative methods such as learned positional embeddings have been explored in the literature [54][50]. Fixed encodings offer superior generalisation for unseen sequence lengths, thus this was the preferred choice during experimentation in *Section 4*. They also eliminate the risk of positional overfitting during training [53].

Having temporally annotated the patch embeddings, the model is now ready for deeper temporal interaction which is first introduced in the proceeding *in-network* prompting stage.

2.2.3 In-Network Prompt

Following the integration of temporal positional encodings, the sequence of patch embeddings progresses to the video encoder stage where **ActionCLIP** begins to model spatial and temporal relationships explicitly [22]. Up to this point, the patch embeddings serve as isolated visual tokens with temporal metadata attached, but they remain unaware of both their spatial neighbours and cross-frame dynamics [41][22]. The video encoder addresses this by employing a combination of ViT layers for spatial modelling and *Temporal Shift Modules* (TSM) for temporal propagation [56][39].

ActionCLIP leverages a pretrained ViT as its base image encoder h_I [22]. Pretraining is performed on large image-text datasets like Kinetics 400 [57], which equips the encoder with visual semantics, but this step is inherently modality-specific to static images that lack temporal dynamics. Consequently, ViT requires augmentation for motion understanding, while it excels at identifying spatial relationships [46][56].

The input to the ViT is the sequence of patch embeddings $\boldsymbol{Z}^{pos}$ that has previously been enriched with positional encodings following its last stage [41]. ViT operates by computing *self-attention* across these embeddings to model intraframe dependencies [53].

$$Q = \boldsymbol{Z}^{pos} W_Q, \quad K = \boldsymbol{Z}^{pos} W_K, \quad V = \boldsymbol{Z}^{pos} W_V \tag{13}$$

With learned projection matrices $W_Q,\ W_K,\ W_V \in \mathbb{R}^{D \times d_k}$ from Equation (13), the attention score between patch i and patch j quantifies their relevance [58] via Equation (14).

$$\alpha_{ij} = \frac{\exp\left(\frac{Q_i \cdot K_j^T}{\sqrt{d_k}}\right)}{\sum_l \exp\left(\frac{Q_i \cdot K_l^T}{\sqrt{d_k}}\right)} \tag{14}$$

These scores are normalised via *softmax* [53], entailing that attention weights across all patches sum to one. The output embedding for patch i is computed with Equation (15) [59].

$$\boldsymbol{z}_i^{out} = \sum_j \alpha_{ij} V_j \tag{15}$$

Initially independent patches, after this step, can now become aware of their spatial context relative to one another [60] and the model can use that information to interpret their structure such as edges, textures, and object boundaries between each frame [41].

Despite this enhancement, ViT remains *spatially bound* – it does not propagate information across time, so an additional module is required to recognise motion patterns [46][21].

To incorporate time with space, **ActionCLIP** integrates TSMs between ViT layers as displayed in Figure 6 [22]. TSM enables *cross-frame communication* by rearranging patch embedding channels along the temporal axis without introducing additional parameters [61][56].

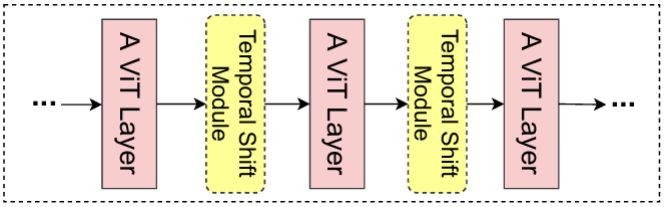


Figure 6: Alternation of Visual Transformer (ViT) and Temporal Shift Module (TSM), resulting in a spatiotemporally aware embedding

Conceptually, for each patch embedding at time step t, TSM partitions its channel dimension into three segments and applies shifts. A forward shift in time from $t-1$ to t, and unchanged portion that stays in t, and a backward shift in time from $t+1$ to t [56][62], refer to *Appendix A.3* for an annotated example. Formally, this operation can be represented by Equation (16).

$$\boldsymbol{Z}_t^{shifted} = shift\ left(\boldsymbol{Z}_t) + \boldsymbol{Z}_t + shift\ right(\boldsymbol{Z}_t) \tag{16}$$

By redistributing feature channels across time, TSM layers allow motion cues to propagate between frames, hence the network is able to build an implicit temporal receptive field that successfully detects motion patterns such as velocity and trajectory of visual elements across frames [56]. The implementation of TSM complemented with ViT in an alternating stack aims to achieve progressive fusion of spatial and temporal information, where each patch embedding encodes both spatial layout and motion dynamics to form spatiotemporally aware representations [56][21].

After processing through the stacked ViT and TSM layers, the *motion-aware patch embeddings* are aggregated into *frame-level embeddings* $\boldsymbol{F}_t$ [22]. This is typically achieved via *mean pooling* over the spatial dimension [41], described by Equation (17).

$$\boldsymbol{F}_t = \frac{1}{N}\sum_{(x,y)} \boldsymbol{z}_{(x,y,t)}^{out} \tag{17}$$

Where N is the total number of patches per frame.

The output sequence $\boldsymbol{F} = \{\boldsymbol{F}_t\}_{t=1}^{T}$ represents the *temporally-aware frame embeddings* [21][46] that are ready for further conditioning in the *post-network* prompting stage.

2.2.4 Post-Network Prompt

Upon completion of the alternated ViT and TSM layers, the sequence of frame-level embeddings $\boldsymbol{F}$ requires final consolidation. This is because while they encode rich localised information about individual frames, **ActionCLIP** necessitates a global video-level representation $g_v \in \mathbb{R}^D$ to facilitate *cross-modal alignment* with the corresponding textual embedding g_t [44][63], which is discussed in *Section 2.2.5*. This is the exact role of the *post-network prompt* – a decisive refinement stage designed to summarise the complete video narrative into a compact, discriminative vector [22][64].

ActionCLIP provides architectural flexibility at this stage by supporting multiple aggregation strategies desired for different complexities of the target video data [22][65]. The first of which is *mean pooling*, being the most computationally efficient approach wherein frame embeddings are averaged along the temporal axis [41], computed via Equation (18).

$$g_v = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{F}_t \tag{18}$$

This method is void of parameters and fast, making it attractive for scenarios where inference speed is critical [56]. On the down side, *mean pooling* treats all frames equally, thus discarding finer temporal order and dynamics [65]. It suffices for videos where overall content predominates over temporal sequence, such as static or repetitive movements, but would perform poorly on modelling long-range dependencies between frames.

For more effective temporal understanding, **ActionCLIP** supports *Temporal Convolution* (Conv1D) or *Long-Short Term Memory* (LSTM) networks applied over the sequence of frame embeddings [22]. Conv1D works well for modelling information in a relatively close time frame, with the efficacy depending on the kernel size [66].

The Conv1D layer applies convolutional filters along the temporal dimension, capturing local frame-to-frame dependencies [32] . Thus when adjacent frames relay information sequentially, it is easy for the layer to recognise an action [67]. If an initial frame contains important information that relates to a distant frame, this poses as a challenge for Conv1D to perform action recognition [32]. A preferable method is to use LSTM [22], which

introduces *recurrent memory* to permit the model of capturing long-range temporal patterns and dependencies [39]. This is particularly valuable in surveillance scenarios where a build up of actions may progressively lead to a crime [68], like *"loitering leading to theft"*.

The internal gating mechanisms of LSTM – involving input, forget, and output gates – dynamically control the flow of information across time steps and allow selective preservation of temporally salient features [18].

With all these methods seemingly reliable enough, the most sophisticated option is the *Temporal Transformer* [22] where frame embeddings are projected into queries Q, keys K, and values V, easing pairwise interaction between frames. Eventually the model is able to capture global temporal dependencies that attend to both local motions and long-range frame relationships without sequential bias [69]. This would entail that they are advantageous in videos with non-linear temporal patterns, abrupt scene transitions, or concurrent actions unfolding asynchronously across different segments [70].

Regardless of the aggregation strategy deployed (see *Appendix A.4* for a detailed comparison between methods), the outcome of the *post-network prompt* is a unified, semantically abundant video embedding $g_V \in \mathbb{R}^D$. This vector encapsulates the temporal evolution and spatial content of the input video, serving as **ActionCLIP's** final video representation which is fed into the cross-modal similarity calculation pipeline along with the textual embedding g_t. Together, they undergo contrastive learning to align the visual content with its textual description [22].

2.2.5 Label Pipeline and Classification

Having produced the video embedding, **ActionCLIP**'s parallel objective is to construct a corresponding *textual embedding* $g_t \in \mathbb{R}^D$ that semantically represents each candidate action class [22][71]. This is the second part to the shared embedding space needed for classification.

Unlike conventional classification models that rely solely on discrete class labels, **ActionCLIP** augments class names with natural language prompts that provide deeper semantic context [72][73]. This is achieved by constructing prompts from three components; (i) *prefix prompt* P_{pre} – optional phrase placed before the class name, *e.g.,* "a video of". (ii) *cloze prompt* P_{clo} – the core action label, *e.g.,* "shoplifting". (iii) *suffix prompt* P_{suf} – optional descriptive phrase appended after the class name, *e.g.,* "in a crowded store" [49][74].

The final textual input is the concatenation of these components, $T = [P_{pre}; P_{clo}; P_{suf}]$ to form the prompt *"a video of shoplifting in a crowded store"* [44]. This prompt engineering strategy enables **ActionCLIP** to capture subtleties in action semantics and generalise to nuanced behaviours [22], a positive point of value in surveillance cases where context typically informs intent [75].

The textual prompt T is then processed by the *pretrained text encoder* g_W, originally adapted from CLIP's transformer-based text encoder [44][49]. The encoder uses self-attention scores to capture contextual relationships between tokens and outputs a single, dense *text embedding* $g_t = g_W(T), \in \mathbb{R}^D$ [53]. This final embedding represents the entire prompt, including both action and context.

With both video and text embeddings available, **ActionCLIP** computes their similarity using a scaled dot product [22] as shown in Equation (19).

$$s(v,t) = \frac{\langle g_v, g_t \rangle}{\tau} \tag{19}$$

This scaling ensures that the logits are well calibrated prior to *softmax normalisation*, a useful step for stable contrastive learning [76]. Higher similarly scores indicate stronger alignment between the visual content and the semantics of the textual prompt, effectively operationalising *zero-shot action recognition* [22].

ActionCLIP is trained using a contrastive loss function denoted by Equation (8) that jointly optimises video-to-text and text-to-video alignment objectives [22], with the probability of a given text prompt t conditioned on a video v given by Equation (20). Symmetrically, the probability of video v given text t is provided by Equation (21).

$$P(t|v) = \frac{\exp(s(v,t))}{\sum_{t'}(s(v,t'))} \quad (20), \qquad P(v,t) = \frac{\exp(s(v,t))}{\sum_{v'}(s(v',t))} \tag{21}$$

The total training objective combines both directions, see Equation (22). This bidirectional alignment encourages the model to produce high similarity scores for correct video-text pairs while simultaneously suppressing mismatched pairs [77][78][22].

$$\mathcal{L} = -\frac{1}{2}(\log P(t|v) + \log P(v|t)) \tag{22}$$

During inference, **ActionCLIP** performs classification by computing the similarity between the video embedding g_v and all candidate text embeddings $\{g_t^k\}_{k=1}^K$, selecting the class with the maximum similarity [22] using Equation (23).

$$\hat{t} = \arg\underbrace{\max}_{t}\ s(v,t) \tag{23}$$

Equation (23) allows **ActionCLIP** to operate in a fully open vocabulary setting, hence [80] the inclusion of new actions at test time can simply be brought upon without additional retraining.

Through prompt engineering, joint embedding learning, and contrastive optimisation, the model transforms discrete class labels into contextual prompts and is able to align them with their respective video representations [49][22]. This architecture provides enhanced action recognition in supervised scenarios and also empowers the network to recognise unseen classes, and therefore applies a unique flexibility to HAR where behaviours continuously evolve.

3 Project Background

The goal of this project is to advance the application of multimodal learning frameworks for action recognition in surveillance contexts. Specifically, this study investigates the versatility and performance of our **ActionCLIP** model across two complementary experimental objectives that were originally structured within a defined project plan in *Appendix C.1*.

The first objective focuses on the reproduction of the official **ActionCLIP** performance on the UCF-101 dataset, a human action recognition benchmark comprising 13,320 video clips across 101 action categories [88]. To rigorously assess **ActionCLIP**'s few-shot learning capability, we implement and evaluate the model across *1-shot*, *2-shot*, and *4-shot* training regimes. These experiments were scheduled for early execution as depicted in *Appendix C.2*, forming the foundational layer of the project timeline. Their purpose was to validate the core architecture and training pipeline, to ensure technical consistency with published benchmarks, and to provide a reliable baseline for further customisation.

The second objective builds upon this foundation by fine-tuning **ActionCLIP** on the UCF Crime dataset, a real-world surveillance dataset containing over 1,900 untrimmed videos spanning 14 anomalous event categories [89]. Unlike curated datasets such as UCF-101, UCF Crime exhibits the visual complexity and motion ambiguity of surveillance environments. The goal is to assess whether **ActionCLIP**'s multimodal pretraining and open-vocabulary nature confer advantages in detecting atypical behaviours in noisy, real-world footage. Fine-tuning on UCF Crime also evaluates **ActionCLIP**'s adaptability to surveillance-specific domains, contributing practical insights into its deployment potential in security infrastructures.

While both objectives were initially envisioned as part of a GPU-accelerated workflow executed on the **University of Southampton**'s *Iridis 5* HPC cluster, unexpected administrative delays in gaining GPU access led to a project revitalisation. The revised implementation adopted a CPU-only training regime, and the project timeline was adjusted to prioritise dataset preparation, model adaptation, and experiment redesign. These changes are documented in the updated Gantt chart and risk assessment in *Appendix C.2* and *Appendix C.3* respectively, which reflect how project planning evolved in response to unforeseen technical and institutional constraints.

The objectives are underpinned by the following research hypotheses: (i) ***ActionCLIP** will successfully reproduce few-shot learning benchmarks on UCF-101 in the 1-shot, 2-shot, and 4-shot regimes.* (ii) *Fine-tuning **ActionCLIP** on UCF Crime will enhance its ability to recognise surveillance-related anomalous behaviours*.

The two objectives are architecturally and methodologically connected to form a sequential pipeline of events: initial reproduction of known results to validate model integrity, followed by domain adaptation to test surveillance applicability. Starting from a small background and low experience in machine learning, it was mandatory to plan the project in two main stages – the first spanning between the months of October to January, undertaking several small scaled projects experimenting with *Multilayer Perceptron* (MLP), CNNs, and Transformers on the MNIST dataset whilst simultaneously learning key concepts in computer vision and deep learning through university modules and a UDEMY course on machine learning (see *Appendix A.5*). The second spanned the months of February to May, focusing mostly on the implementation of **ActionCLIP** as detailed in this report.

4 Experimental Design

4.1 Dataset Selection

A successful action recognition model is one that is deeply intertwined with the quality, relevance and diversity of its training data. Given the dual objectives of this project, two datasets were strategically selected to both benchmark reproduction and adapt to the real world. UCF-101, representing standardised human action categories, and UCF Crime, used for fine-tuning and representing surveillance orientation.

UCF-101 is a canonical dataset in the action recognition community, comprising of short video clips drawn from YouTube and encompassing 101 human action categories such as *Apply Lipstick* and *Tai Chi*. Videos are trimmed to contain a single action class and exhibit a variety of quality features – like diversity in illumination conditions and camera motions. Using official splits, training samples are constrained to *1-shot*, *2-shot*, and *4-shot* regimes to emulate low-resource scenarios, which enables direct comparability with **ActionCLIP**'s published results and establishes a controlled environment for validating the implementation.

Upon searching for a surveillance dataset to support the fine-tuning objective of this project, multiple publicly available crime-focused datasets were considered. These included SPHAR, SAD, DIVA, XD-Violence, Avenue, ShanghaiTech, and StreetScene – all of which offer real-world or synthetic surveillance footage with various levels of annotation and granularity. However, each presented ethical, legal, or logistical limitations that rendered them impractical under the scope and constraints of this research. The final choice of UCF Crime was grounded in its *Open Licensing* (CC0) and lack of *Personally Identifiable Information* (PII). Table I summarises the comparison between the evaluated datasets and justifies the ethical and legal rationale for selecting UCF Crime.

Table I: Legal, ethical, and rational comparison between surveillance datasets

Dataset	License & Access	Ethical Constraints	Use Case Fit	Limitations
UCF Crime	Public (CC0), Kaggle	Anonymised, no PII	Excellent - 14 labelled crime classes	Long untrimmed videos, class imbalance
SPHAR	Licensing per class required	Real surveillance, possible PII	Good – 35 classes, 3 views	Unscalable licensing; ethics clearance complex
SAD	Requires approval via request	Sensitive actions, poor documentation	Moderate – 6 actions	Ethics unclear; poor access workflow
DIVA (DARPA)	Restricted, approval required	National-level dataset, subject to export laws	Excellent coverage	Government clearance needed
XD-Violence	Non-commercial use only	Contains violent/disturbing content	Good for anomaly detection	Careful content handling needed
Avenue	Academic use permitted	Campus surveillance, anonymised	Weak – no class labels	Too small (16 videos)
ShanghaiTech	Free with citation	Urban surveillance with anonymisation	Weak – only frame-level anomalies	No categorical class structure

StreetScene	Research use with citation	Public vehicle/pedestrian footage	Niche – scene-level violations	Granularity mismatch; not individual action-focused

UCF Crime is a large-scale, untrimmed video dataset explicitly curated for anomaly detection in surveillance footage. In comparison to UCF-101, UCF Crime presents significant real world challenges, one of which is class imbalance within its 14 classes – normal activities (within the class *NormalVideos*) vastly outnumber criminal activities (videos from the 13 other classes). Downloading this dataset from Kaggle [91], a popular dataset repository, revealed that frame sampling had already been done by the original distributor, where each video had undergone frame extraction.

Given the sensitive nature of surveillance footage, particularly involving real criminal incidents, ethical compliance was required. An *ERGO II* ethics application was submitted and approved by the **University of Southampton**'s Ethics Committee, viewable in *Appendix D.1*.

Key ethical protocols included the use of publicly available datasets, where UCF-101 and UCF Crime are both open-access datasets with no PII retained in research outputs, as well as anonymisation and responsible handling, where dataset storage and access are restricted solely to the research conductor for privacy and data protection. In addition, the consideration of purpose limitation, where usage is confined strictly to research activities described in the approved ethics application.

For compatibility with the **ActionCLIP** training framework, both the UCF-101 and UCF Crime datasets were subjected to a preparation pipeline with the objective of transforming raw video data into structured frame sequences. The complete workflow is illustrated in Figure 7.

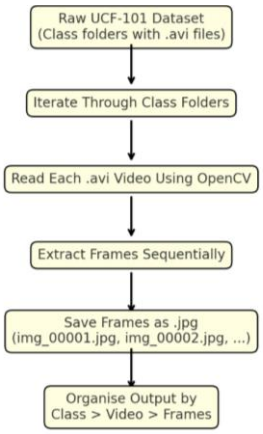


Figure 7: Flowchart of dataset preparation used for UCF-101 and (partially) UCF Crime

The process in Figure 7 begins with the acquisition of video files sourced from the official distributions of UCF-101 [90] and UCF Crime [91]. For UCF-101, the dataset is originally composed of class-labelled **.avi** video files. To prepare these for **ActionCLIP**, a custom frame extraction script was developed using the OpenCV Python library. The script recursively traverses the UCF-101 directory structure, identifies each video, and decodes it frame by frame without skipping or subsampling, see *Appendix B.2* for the frame extraction script. All frames are extracted in full, preserving the native temporal resolution of each video. The frames are saved as sequentially numbered **.jpg** files, *e.g.*, **img_00001.jpg**, **img_00002.jpg**, in structured output folders organised by class and video name. No resizing or resolution normalisation is applied at this stage, thereby maintaining

the original spatial aspects of the dataset. This approach maintains complete representation of motion in each clip.

To generate validation data for model tuning and benchmarking, a script available in *Appendix B.3* was used. This script performs *class-wise stratified sampling* to split a fixed percentage of videos into a validation set. Specifically, a 5% validation ratio is applied to each class. The script reads class labels from **ucf_labels.csv**, matches them to folder names within the frame directory, and randomly selects the appropriate number of videos per class. It then outputs a manifest file, **val_rgb_split1.txt**, where each line contains a relative path to a video's frame directory and its corresponding numeric class label. This file is directly usable by **ActionCLIP's** dataloader to ensure retrieval during validation runs.

Following the discussion of the first objective to replicate **ActionCLIP**'s performance under few-shot learning conditions, the training splits were constructed for *1-shot*, *2-shot*, and *4-shot* settings via the UCF-101 dataset. Each split was generated such that only 1, 2, or 4 video instances per class were available during training.

The input to the model consisted of 8 temporal segments per video to allow direct comparability to **ActionCLIP**'s results, each segment contributing one frame. This configuration ensures that the model receives temporal diversity even in extremely data-constrained settings.

To maintain clarity between experimental conditions, each few-shot experiment was trained independently. Weights were not carried over between the *1-shot, 2-shot,* and *4-shot* runs. In each case, while the CLIP backbone was initialized with pretrained weights, the **ActionCLIP** specific layers such as the fusion module were randomly initialized. This isolation ensures that performance differences reflect the impact of data quantity alone, without contamination from prior training. All experiments were run using the same model backbone and training pipeline, with a complete overview of the configuration provided in Table II.

Table II: Configuration of *1-shot, 2-shot,* and *4-shot* training regimes ran on CPU implementation

Parameter	1-Shot	2-Shot	4-Shot
Architecture	ViT-B/16	ViT-B/16	ViT-B/16
Batch Size	4	4	1
Segments	8	8	8
Input Size	224x224 px	112x112 px	112x112 px
Augmentation	None	None	None
Epochs	50	30	10
Loss Function	KL Divergence	KL Divergence	KL Divergence
Optimiser	AdamW	AdamW	AdamW
Learning Rate	5×10^{-6}	5×10^{-6}	5×10^{-6}
Training Time	36:22:05	38:50:32	40:09:46

4.2 Implementation of ActionCLIP: Methodology

4.2.1 Fine-Tuning Method

The first implementation of **ActionCLIP** for this project was based on a modified version of the official opensource repository [92]. As previously mentioned, the original method was planned to be executed using the **University of Southampton**'s HPC cluster, *Iridis 5*, but due to administrative delays in gaining approval for GPU resource access, the method was adjusted to support CPU-based training and fine-tuning. The core implementation consists of a modular training pipeline, YAML-based configuration (see *Appendix B.7, B.8, B.9, B.10*) management, and updated scripts for training and evaluation.

The training routine is orchestrated by a script, train.py detailed in *Appendix B.5*, which loads configuration parameters from a YAML file to construct the training environment. This training loop involves frame-level image preprocessing, temporal fusion through the prompt module, and contrastive loss computation against text embeddings derived from tokenised class prompts. All components have been made compatible with CPU execution – though they are also compatible with GPU, with floating-point precision adjusted conditionally based on device availability.

The evaluation pipeline is similarly structured within the testing script in *Appendix B.6*, where inference is performed on the validation split using the same data loader and model checkpointing logic. Predictions are compared against ground truth using standard classification metrics including Top-1 accuracy and Top-5 accuracy.

To ensure reproducibility and cross-platform compatibility, the project environment was encapsulated using the file found in *Appendix B.1*. This file defines all dependencies, including PyTorch 1.8.1, torchvision, scikit-learn, OpenCV, WandB for logging, and DotMap for configuration parsing. Additionally, it installs CLIP-compatible libraries such as ftfy, regex, and tqdm, as well as video processing utilities like ffmpeg and imageio. The environment was also modified to support both CPU and GPU execution, since this implementation was run exclusively on a CPU-based system.

Training runs are automatically logged to *Weights & Biases* (WandB), where total loss, image and text losses, and learning rate are tracked per batch. Checkpoints are saved after each epoch and include model and fusion model states. WandB also logs and displays confusion matrices and classification reports during evaluation.

The initial plan for the second objective was to fine-tune **ActionCLIP** on all 14 classes of the UCF Crime dataset, allowing the model to classify any given video into its corresponding anomalous or normal category. This would have enabled a direct application of **ActionCLIP** as a multiclass surveillance activity recogniser. However, this approach proved infeasible under the project's hardware constraints. Training a full transformer-based model across 14 classes and tens of thousands of untrimmed frames using only CPU resources yielded an estimated training duration of 753.24 days. Downscaling the training set would have undermined the distributional integrity of each class, resulting in under-represented classification [93]. As such, this approach was abandoned early and mitigated according to the risk assessment compiled in *Appendix C.3*, but it established a baseline understanding of the computational demand involved.

Thus, to reduce the problem's complexity, the next attempt reframed the task as binary classification – distinguishing between *NormalVideos* and *Shoplifting*. This plan

preserved the fine-tuning mechanism, training **ActionCLIP** using 10,000 frames per class and evaluating it using 2,000 unseen frames per class. Since UCF Crime was provided in the form of pre-extracted image frames rather than raw video files, preparation began at the frame organisation stage. The script in *Appendix B.4* was written to automate both label mapping and data indexing. This script generates two files: the first of which assigns numeric labels to each class, *e.g.,* 0 for *NormalVideos* and 1 for *Shoplifting*. The second file lists the relative path to each frame along with its corresponding label for **ActionCLIP**'s fine-tuning. The script in *Appendix B.3* traverses the image directories, detects **.png** frame files, and formats them in accordance with **ActionCLIP**'s expected input style. This streamlined the ingestion of surveillance frame data into the **ActionCLIP** training pipeline and ensured reproducibility of class mappings across training sessions.

Despite the simplified setting, the model achieved only 28.18% accuracy – below random chance for binary classification. This result suggested that either the text encoder was failing to properly align class prompts to video features, or the visual encoder lacked sufficient capacity to generalise to the surveillance domain. It became necessary to investigate each modality independently.

4.2.2 Prompt Engineering Method

Following the underperformance of **ActionCLIP** during binary classification on the UCF Crime dataset, and given the failure occurred in a setting with only two well-defined classes, attention was drawn toward the model's semantic alignment mechanism: the ability of the textual prompt to meaningfully anchor the video representation during inference.

This hypothesis was grounded in one of **ActionCLIP**'s core design principles – that classification occurs not through fixed class labels, but via text-video similarity within a shared embedding space [22]. In this paradigm, class representations are not simply one-hot encoded categories; they are language prompts whose phrasing directly affects the vector space the model compares against. As noted in the original CLIP paper and later reinforced in zero-shot transfer works like CoOp and CoCoOp, prompt formulation can heavily bias classification outcomes [49][94].

In the context of UCF Crime, the default prompt design used in the initial experimental plan was semantically vague; the simple class names *Shoplifting* and *NormalVideos* do not offer the model enough grounding to distinguish between subtle, temporally extended behaviours. Moreover, CLIP's pretraining data did not include surveillance or CCTV footage [44].

Rather than attempting to retrain or fine-tune the text encoder – which would be computationally intensive and against the constraints of this project – a more feasible alternative was to manipulate the prompts themselves. This approach, known as prompt engineering, is a lightweight but effective form of intervention that alters the textual input without modifying the model's parameters.

From an experimental design perspective, prompt engineering is a modality-isolated test as it requires no gradient computation, enables direct manipulation of the input text space, and could be applied retrospectively meaning using the same video embeddings and testing multiple prompt sets over a single run [95]. Furthermore, a positive result from this method would allude that the initial plan's failure was not entirely visual, but at least partially due to insufficient textual grounding.

It appeared that default class-level prompts, which simply tokenised the action names, lacked sufficient semanticity to guide the model's cross-modal alignment in a noisy dataset. This inspired a series of tests using more descriptive and contextually relevant prompts, including full sentence formulations seen in Table III. The strategy of prompt ensembles, inspired by *Radford et al. (2021)* [44], was to average the encoded embeddings of multiple semantically equivalent prompts for each class, thereby producing more adaptable textual representations. This technique, incorporated by the process displayed in Figure 8, had proven highly effective in zero-shot classification tasks, and it was hypothesised that it could significantly improve alignment for low-signal surveillance frames as well. By replacing the class prompts fed into the model with pre-tokenised baseline, descriptive, and surveillance-context prompts, we used CLIP's tokenise method to convert these prompts into embeddings, then normalised them and ran inference through the script provided in *Appendix B.11*. We also ran inference for each individual prompt type to distinguish between **ActionCLIP**'s preferred style.

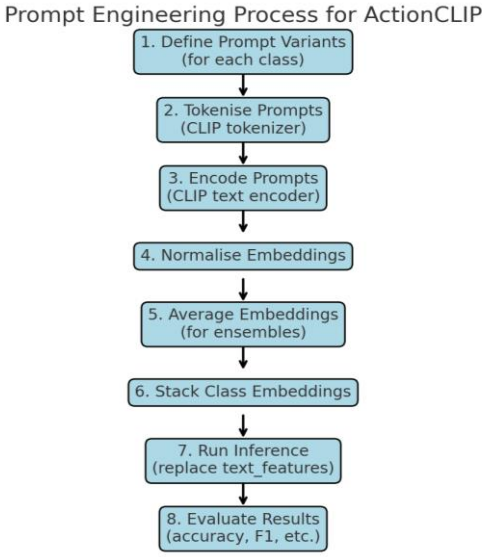


Figure 8: Flowchart of prompt engineering steps, coded in *Appendix B.11*

To further explore this space, cloze templates were evaluated to test whether implicit label anchoring improved grounding. Finally, negation-based prompts were introduced as a form of semantic contrast, with the aim of sharpening decision boundaries by explicitly encoding absence-of-action cues.

Table III: Prompt types used in prompt engineering experiments, given with an example

Prompt Type	**Example**
Baseline	A video of Shoplifting
Descriptive	A person is Shoplifting in a store
Surveillance-Oriented	CCTV footage showing a Shoplifting incident
Ensemble	*Average embeddings of the three above*
Cloze	This is a video of someone Shoplifting
Negation	A video where nothing illegal happens

4.2.3 Anomaly Detection Method

After exploring prompt engineering, it became evident that improvements from language-side intervention – while significant – still left the model far below practical utility. Even with prompt ensembles, classification accuracy in the binary *Shoplifting* vs *NormalVideos* task stabilised at 44.13%. This raised a critical follow-up question: if modifying the language input yields diminishing results, could the underlying limitation lie instead in how the visual encoder perceives and represents the input video?

To isolate and evaluate this component independently, the project moved toward a fundamentally different formulation; removing the classification mechanism altogether and testing whether **ActionCLIP**'s video encoder on it own could detect anomalous behaviour without any supervision or linguistic guidance. This motivated the third experiment, reframing the task to unsupervised anomaly detection.

The logic behind this decision is grounded in the nature of the UCF Crime dataset. Real-world surveillance footage often contains unlabelled or weakly-labelled events, highly variable scenes, and behaviours that do not fit neatly into closed-world class categories. Attempting to teach a model to categorise such data using explicit labels and prompts presupposes the structure we aim to discover. In contrast, anomaly detection techniques allow the model to operate in an open-world setting, learning only from what "normal" looks like, and flagging anything significantly different as anomalous.

This approach employed *Mahalanobis distance* seen in Equation (24). By computing the mean embedding vector of all *NormalVideos* and modelling their spread using a full covariance matrix, each test embedding could be scored based on how far it lies from this learned "normality boundary". Samples that exceeded a calibrated distance threshold of 95th percentile were flagged as anomalous.

$$d(x) = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)} \tag{24}$$

First, all video embeddings $g_v \in \mathbb{R}^D$ were extracted from the training set of *NormalVideos* using the **ActionCLIP** video encoder with the number of segments set to 8. From these, a mean vector $\mu \in \mathbb{R}^D$ and a full covariance matrix $\Sigma \in \mathbb{R}^{D \times D}$ were computed, defining the statistical centroid and spread of normal behaviour in the space.
Each validation sample was then passed through the same visual encoder, and its Mahalanobis distance from the distribution was computed. Thus, samples with high distance are flagged as anomalies under the assumption that all other class embeddings would deviate significantly from the learned representation of normality. These preceding and proceeding steps are included in *Appendix B.12*. A threshold was set by evaluating the distance distribution on held-out *NormalVideos* and selecting the 95th percentile as a cut-off. Any validation sample exceeding this threshold is classified as *Anomalous*.
This methodology offers multiple advantages. Firstly, it allows the model to function as a zero-shot anomaly detector, requiring no retraining and no textual inputs. Secondly, it operates fully on the visual encoder, thus providing a clean, modality-isolated evaluation of how well **ActionCLIP** encodes semantic deviation. On the other hand, this model is only applicable to the field of surveillance due to its *Normal* vs. *Anomalous* binary nature. More importantly, this method directly addresses the central research hypothesis: whether **ActionCLIP**'s pretrained knowledge could generalise to real-world, high-noise surveillance footage without relying on labelled examples of anomalous behaviour. In a practical surveillance context, this approach is operationally realistic since anomalies are often unpredictable and unlabelled, yet it remains critical to detect them based on deviation from expected patterns.

5 Results and Evaluation

This section presents the results of both experimental objectives, run on CPU using the **ActionCLIP** architecture and configurations described in *Section 4.1* and *Section 4.2*. The results are grouped accordingly and interpreted in light of each objective's goal.

5.1 Few-Shot Learning on UCF-101

The three training runs on *1-shot*, *2-shot*, and *4-shot* learning using UCF-101 were initialised from CLIP's pretrained weights and evaluated using Top-1 and Top-5 accuracy on a held out 5% validation split. The results are summarised in Table IV.

Table IV: Top-1 and Top-5 accuracies obtained in 1-shot, 2-shot, and 4-shot training on UCF-101

Shot Count	Top-1 Accuracy (%)	Top-5 Accuracy (%)
1-Shot	60.00%	85.69%
2-Shot	62.19%	88.38%
3-Shot	63.61%	89.04%

These results indicate a consistent improvement in performance with the increase in the number of shots, demonstrating the model's ability to leverage additional data effectively. To contextualise these findings, we compare our results with those reported in the **ActionCLIP** paper [22] and other state-of-the-art models using Figure 9.

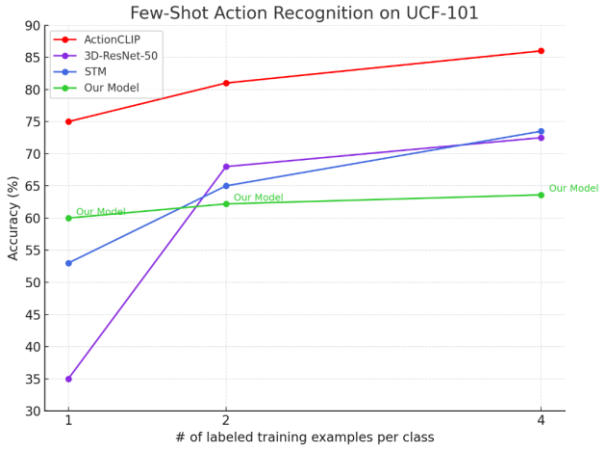


Figure 9: Resultant accuracy for 1-shot, 2-shot, and 4-shot comparison between **ActionCLIP**, 3D-ResNet-50, STM, and this paper's **ActionCLIP** implementation. Values taken from [22]

Our 1-shot Top-1 accuracy of 60.00% falls short of the original **ActionCLIP**'s reported ≈ 75%, indicating areas for improvement in our implementation, but most likely in our training regimen considering the computational resources used. Compared to STM and 3D-ResNet-50, our CPU implementation of **ActionCLIP** outperforms in low-resource implementations.

Models like HyRSM and HCR using ResNet-50 backbone exhibit significantly higher accuracies of 83.9% and 82.4% respectively, but they rely on much heavier pretraining than our CPU implementation [47].

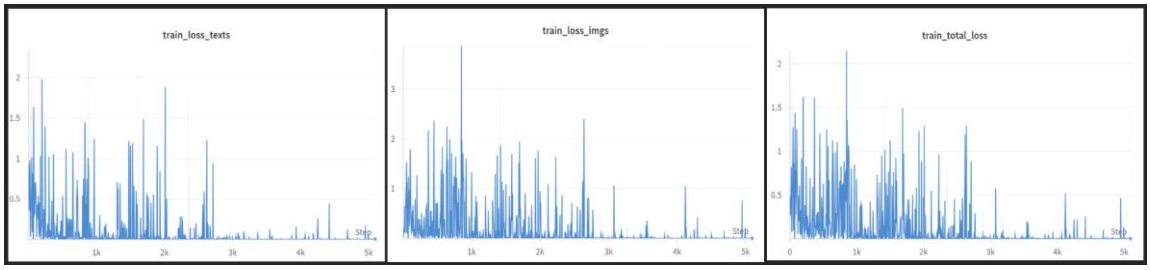


Figure 10: The figures represent the 1-shot regime's (a) Left: Text-branch loss during training, (b) Middle: Image-branch loss during training, (c) Right: Total training loss from both text and image branches combined

To further support the training dynamics observed during experimentation, we present the loss curves for the text branch, image branch, and the total combined loss throughout training on the 1-shot implementation. These are visualised in Figures 10 (a), (b), and (c).

Figure 10 (b) shows the evolution reflecting the contrastive loss between the video embeddings and their corresponding class prompts. The curve displays significant variance during the early training stages, a common trait in few-shot learning where the model begins from a weakly-informed prior and must rely on limited supervision to shape meaningful representations. This phenomenon is called instability in early-phase few-shot learning, and is the result of high gradient sensitivity as the model searches for discriminative features with minimal support examples. However, a sharp downward trend can be seen after approximately 2,800 iterations, indicating rapid adaptation and convergence with slight outliers. These outliers are prominent when the weight update step causes the weights to deviate largely from the model's best, possibly due to a high learning rate.

In parallel, Figure 10 (a) shows the text-related loss, which demonstrates a similar trend. The text branch shows more consistent stability during mid- to late-phase training compared to the image stream. This may reflect the pretrained nature of CLIP's textual encoder, which likely requires less representational tuning compared to the visual encoder that must adapt to new video frame distributions.

The aggregate metric of total loss presented in Figure 10 (c) reinforces the progressive minimisation of contrastive loss between the video and textual modalities. The loss flattens toward zero as training stabilises, showcasing the KL-divergence based optimisation and proper alignment of modalities within the embedding space.

5.2 Fine-Tuning on UCF Crime

Following few-shot benchmarking on UCF-101, the best performing **ActionCLIP** checkpoint was used to initialise fine-tuning on the UCF Crime dataset. Only 5 epochs of training were conducted with a learning rate of 10^{-5} and 2 segments reflecting the computational constraints and the hypothesis that the pretrained multimodal alignment would suffice to transfer representations into the surveillance domain with minimal tuning.

Despite the observations during UCF-101 training, fine-tuning results were underwhelming, with a Top-1 accuracy of 28.18% which is inferior to random guessing in binary classification.

5.2.1 Prompt Engineering Approach

These variants were tested under fixed inference-only conditions, with no fine-tuning, and evaluated based on Top-1 accuracy, precision, recall, and F1-score as seen in Figure 11.

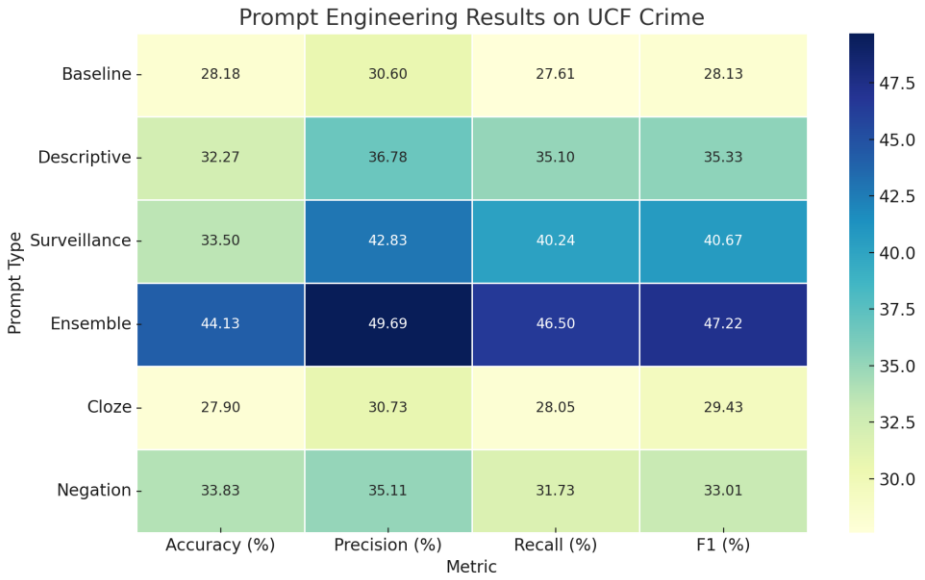


Figure 11: Heatmap comparing the performance of six prompt types across accuracy, precision, recall, and F1 metrics on the UCF Crime dataset

The prompt ensemble strategy improved accuracy to 44.13%, a major gain from language-side intervention.

Table V and Figure 12 present a performance comparison between our handcrafted prompt engineering approach and several state-of-the-art prompt tuning methods evaluated on the DOSCO-2k benchmark. The P-UCF column corresponds to UCF-like human action domains. CLIP, CoOp, and CoCoOp were evaluated on curated, high quality data using ResNet or ViT backbones with extensive GPU-based training [94].

Table V: Overview of model/prompt strategies compared across datasets

Model/Prompt Strategy	**Dataset Type**	**Trainable**
CLIP (Handcrafted Prompt)	DOSCO-2k (Curated)	No
CoOp (Learnable Prompt)	DOSCO-2k (Curated)	Yes
CoCoOp (Instance-Conditioned)	DOSCO-2k (Curated)	Yes
ActionCLIP (our model – Prompt Ensemble)	UCF Crime (Noisy)	No

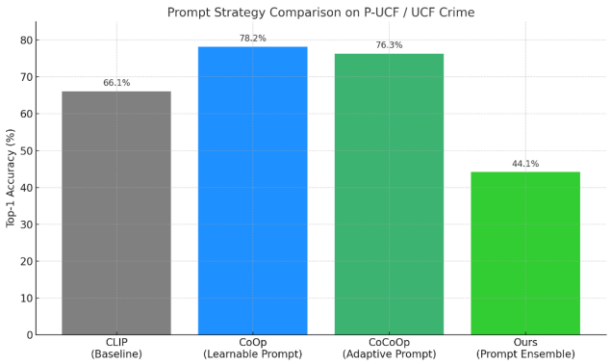


Figure 12: Top-1 accuracy comparison of different prompt strategies, showing higher performance on curated datasets and relative performance of our handcrafted prompt ensemble on the noisier UCF Crime dataset

Compared to state-of-the-art prompt tuning methods like CoOp and CoCoOp, which achieve ~78% accuracy on curated datasets like UCF (P-UCF), our handcrafted prompt ensemble method achieves 44.13% accuracy on the significantly noisier UCF Crime dataset, without any backpropagation or fine-tuning. Although absolute performance is lower, this result is compelling given the absence of GPU training and the domain complexity involved.

5.2.2 Anomaly Detection Approach

The resulting confusion matrix shown in Figure 13 reveals a highly precise classification boundary. Out of 167 samples, 158 were correctly identified, while only 9 were misclassified as *Anomalous*. Conversely, of the 407 anomalous samples, 402 were correctly flagged, with only 5 false negatives. This balance between true positives and true negatives highlights the effectiveness of the statistical threshold in distinguishing outliers in the **ActionCLIP** embedding space. Notably, the low number of false positives suggests that the model learned a robust semantic representation of normality, while the minimal false negatives reflect its ability to identify subtle deviations even without the aid of class-specific prompts.

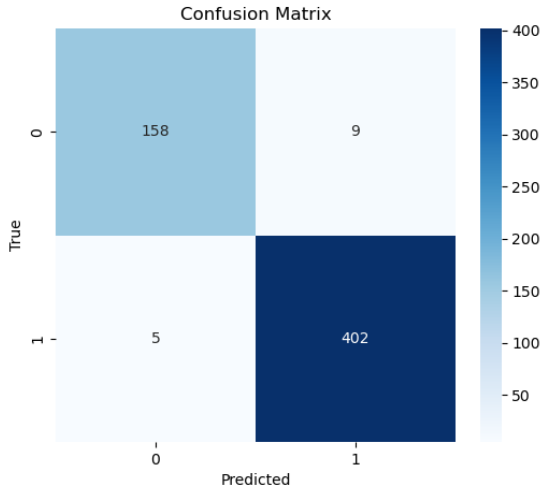


Figure 13: Confusion matrix for the anomaly detection approach, illustrating strong separation between normal and anomalous samples achieved by **ActionCLIP** without class-specific prompts

At the decision threshold of 0.2082, the model achieved an overall accuracy of 97.56%. Precision was measured at 97.81%, while recall reached 98.77%, resulting in a harmonic F1 score of 98.29%. The *Receiver Operating Characteristic's Area Under the Curve* (ROC-AUC) was calculated at 0.9913, indicating a near-perfect ability to separate the two classes in latent space. These metrics suggest that the model's latent space, when projected via Mahalanobis scoring, encodes rich and discriminative information that extends beyond its original training regime since these results are the highest achieved in the experiment plans of this project.

Further insight is provided by the ROC curve comparison shown in Figure 14, which benchmarks **ActionCLIP** against a range of anomaly detection classification baselines including CNN (VGG19) with LSTM, C3D, Inception-V3, and 2-Stream C3D models, as well as two temporal detection methods proposed by Lu et al. and Zhong et al [97].

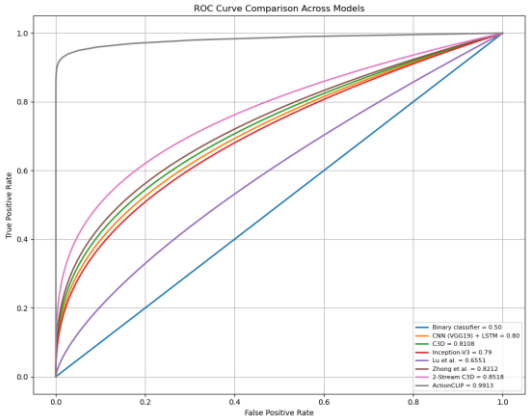


Figure 14: ROC curve comparison across several models with AUC values taken from [97], demonstrating superior **ActionCLIP** discrimination capability with an AUC of 0.9913

The **ActionCLIP**-based anomaly detector significantly outperformed all baseline and state-of-the-art models with respect to the AUC For context, the next best-performing architecture, 2-Stream C3D achieved an AUC of only 0.8518. This difference in performance reinforces the claim that **ActionCLIP**'s pretrained transformer encoder captures a deeper, more structured semantic space suitable for generalised anomaly detection in video.

Qualitative inspection of misclassified samples reveals that the model mislabels *NormalVideos* as *Anomalous* when frame quality is comprised particularly by motion blur and occlusion – features that visually resemble suspicious behaviour. Inversely, some the few *Anomalous* instances missed by the model were visually indistinct from benign interactions, such as the sequence of frames in Figure 15 from the *Stealing* class of UCF Crime, where no actual stealing occurs. The individuals walk up to the vehicle, peek into the front passenger window, and leave the scene without possessing anything from the vehicle itself.



Figure 15: Series of frames from *Stealing* depicting suspicious activity – classified by the model as *NormalVideos*

By human perception this behaviour may look suspicious, but this perspective assumes the individuals in the frames are not owners of the vehicle. Thus by looking at it from a blank slate, the behaviour does not fall under the term "anomalous" without assumption, and some may argue that the model's classification under *NormalVideos* is justifiable.

6 Conclusions

The project set out with the two objectives: (i) ***ActionCLIP*** *will successfully reproduce few-shot learning benchmarks on UCF-101 in the 1-shot, 2-shot, and 4-shot regimes.* (ii) *Fine-tuning* ***ActionCLIP*** *on UCF Crime will enhance its ability to recognise surveillance-related anomalous behaviours*.

The first objective of the project achieved partial success. The **ActionCLIP** model was successfully implemented in a CPU-compatible environment, and while the original paper reported accuracies above 75% even in 1-shot regimes, our implementation achieved a top-1 accuracy of 60% in 1-shot, 62.19% in 2-shot, and 63.61% in 4-shot settings. These lower results still reflect the model's capacity to extract transferable semantic representations under extreme data and computational hardware limitations. Most notably, the findings validate that **ActionCLIP**'s architecture can still operate and generalise in resource-constrained settings, thus validating Objective (i).

The second phase involving fine-tuning the model on a surveillance-oriented dataset encountered greater challenges. The initial plan of fine-tuning the model to all 14 classes of UCF Crime was immediately abandoned when GPU-resources were no longer an option, switching to binary classification using conventional supervised fine-tuning on 2 classes which yielded the poor accuracy of 28.18%. Due to this circumstance, the textual and visual encoders became the focus of the study, investigating the foremost via prompt engineering where accuracy increased by a factor of 1.5 × to 44.13%. This confirms the text encoder's sensitivity to linguistic structure and its capacity to modulate the video-text similarity landscape when properly tuned. However, this improvement remained bounded by the representational limitations imposed by ambiguous action labels and high intra-class variability inherent in the UCF Crime dataset.

In contrast, the anomaly detection experiment eliminated the textual encoder entirely and relied exclusively on ActionCLIP's visual embeddings, modelling semantic deviation via Mahalanobis distance in latent space. The visual-only formulation yielded an ROC-AUC of 0.9913 and an accuracy of 97.56%, an increase by a factor of 3.5 × relative to our baseline classification performance. Such disparity between the prompt-based and vision-only results strongly suggests that the pretrained visual encoder captures a more stable and generalisable representation of spatiotemporal structure, particularly in low-supervision or domain-shifted contexts. These findings support the assertion that within surveillance domains where textual supervision is weak, noisy, or sometimes unreliable, **ActionCLIP**'s visual backbone can serve as a more efficient modality for anomaly detection.

The fact that this performance was achieved without any additional training, and on CPU-only hardware, generously lowers the barrier to deployment in resource-efficient contexts like small retail environments, transportation hubs, or developing regions where edge-based analytics are preferred. The results indicate that pre-existing foundation models like **ActionCLIP** can be adapted for use in lightweight surveillance setups, provided that computationally inexpensive anomaly scoring methods (like Mahalanobis distance) are applied.

However, in terms of reliability for real-time and/or high-stakes surveillance, deployment is not yet recommended. Several factors still limit direct translation of what is considered *anomalous*, including the lack of temporal aggregation in inference and limited interpretability of the embeddings, as well as sensitivity to subtle scene changes that may

not reflect actual threats. As such, while the current model can serve as a somewhat powerful first-line filter or alert system, it should be complemented with manual verification.

One avenue to build upon the present study could explore soft prompt tuning methods such as CoOp or CoCoOp under few-shot learning regimes, especially if GPU resources become available. This would allow the model to learn task-specific linguistic templates and increase alignment performance without full model retraining.

Second, adapting the anomaly detection scoring function could also be refined. Though Mahalanobis distance yielded strong results, it assumes unimodal Gaussian distribution of normal activity. Exploring alternative unsupervised scoring functions – like kernel density estimation, one-class SVMs, or contrastive latent modelling – could yield better performance in multimodal and highly dynamic scenes.

Finally, real-world deployment would benefit from a streamlined inference system with integrated alerting logic, logging, and user interfaces. Coupling **ActionCLIP**'s embedding space with explainability tools like Grad-CAM or CLIP-attention visualisations would allow operators to better trust and interpret system outputs. By combining these enhancements, future implementations can bridge the gap between research prototypes and deployable surveillance intelligence systems.

References

[1] UNODC, “GLOBAL STUDY ON HOMICIDE 2023,” 2023. Available: https://www.unodc.org/documents/data-and-analysis/gsh/2023/Global_study_on_homicide_2023_web.pdf

[2] V. Hugo, C. Henrique, and William Robson Schwartz, “Histograms of Optical Flow Orientation and Magnitude to Detect Anomalous Events in Videos,” *Brazilian Symposium on Computer Graphics and Image Processing*, Aug. 2015, doi: https://doi.org/10.1109/sibgrapi.2015.21.

[3] K. Ohnishi, M. Hidaka, and T. Harada, “Improved Dense Trajectory with Cross Streams,” *arXiv.org*, 2016. https://arxiv.org/abs/1604.08826.

[4] A. Betti, G. Boccignone, L. Faggi, M. Gori, and S. Melacci, “Visual Features and Their Own Optical Flow,” *Frontiers in Artificial Intelligence*, vol. 4, Dec. 2021, doi: https://doi.org/10.3389/frai.2021.768516.

[5] Abdolamir Karbalaie, F. Abtahi, and Mårten Sjöström, “Event detection in surveillance videos: a review,” *Multimedia Tools and Applications*, vol. 81, no. 24, pp. 35463–35501, Jun. 2022, doi: https://doi.org/10.1007/s11042-021-11864-2.

[6] Y.-H. Chen and C.-T. Wu, “ReynoldsFlow: Exquisite Flow Estimation via Reynolds Transport Theorem,” *arXiv.org*, 2025. https://arxiv.org/abs/2503.04500.

[7] R. Gomez-Ojeda, Z. Zhang, J. Gonzalez-Jimenez, and D. Scaramuzza, “Learning-based Image Enhancement for Visual Odometry in Challenging HDR Environments,” *arXiv.org*, 2017. https://arxiv.org/abs/1707.01274.

[8] “HMAFlow: Learning More Accurate Optical Flow via Hierarchical Motion Field Alignment,” *Arxiv.org*, 2015. https://arxiv.org/html/2409.05531v3.

[9] H. Wang, A. Klaser, C. Schmid, and C.-L. Liu, “Action recognition by dense trajectories,” *CVPR 2011*, Jun. 2011, doi: https://doi.org/10.1109/cvpr.2011.5995407.

[10] V. L. Guen, C. Rambour, and N. Thome, “Complementing Brightness Constancy with Deep Networks for Optical Flow Prediction,” *arXiv.org*, 2022. https://arxiv.org/abs/2207.03790.

[11] R. Yamashita, M. Nishio, R. K. G. Do, and K. Togashi, “Convolutional Neural networks: an Overview and Application in Radiology,” *Insights into Imaging*, vol. 9, no. 4, pp. 611–629, Jun. 2018, doi: https://doi.org/10.1007/s13244-018-0639-9.

[12] Z. Li, W. Yang, S. Peng, and F. Liu, “A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects,” *arXiv:2004.02806 [cs, eess]*, Apr. 2020, Available: https://arxiv.org/abs/2004.02806

[13] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, May 2012.

[14] X. Zhao, L. Wang, Y. Zhang, X. Han, M. Deveci, and M. Parmar, “A review of convolutional neural networks in computer vision,” *Artificial Intelligence Review*, vol. 57, no. 4, Mar. 2024, doi: https://doi.org/10.1007/s10462-024-10721-6.

[15] “Deep Video Representation Learning: a Survey,” *Arxiv.org*, 2018. https://arxiv.org/html/2405.06574v1.

[16] “Efficient Temporally-Aware DeepFake Detection using H.264 Motion Vectors,” *Arxiv.org*, 2024. https://arxiv.org/html/2311.10788v2.

[17] A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar, and L. Fei-Fei, “Large-Scale Video Classification with Convolutional Neural Networks,” *2014 IEEE Conference on Computer Vision and Pattern Recognition*, Jun. 2014, doi: https://doi.org/10.1109/cvpr.2014.223.

[18] D. Sun, S. Roth, and M. J. Black, “A Quantitative Analysis of Current Practices in Optical Flow Estimation and the Principles Behind Them,” *International Journal of Computer Vision*, vol. 106, no. 2, pp. 115–137, Jan. 2014, doi: https://doi.org/10.1007/s11263-013-0644-x.

[19] J. Carreira, A. Zisserman, Z. Com, and Deepmind, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset.” Available:

https://openaccess.thecvf.com/content_cvpr_2017/papers/Carreira_Quo_Vadis_Action_CVPR_2017_paper.pdf
[20] C. Feichtenhofer, H. Fan, J. Malik, and K. He, “SlowFast Networks for Video Recognition.” Available: https://openaccess.thecvf.com/content_ICCV_2019/papers/Feichtenhofer_SlowFast_Networks_for_Video_Recognition_ICCV_2019_paper.pdf
[21] G. Bertasius, H. Wang, and L. Torresani, “Is Space-Time Attention All You Need for Video Understanding?” Available: https://arxiv.org/pdf/2102.05095
[22] M. Wang, J. Xing, and Y. Liu, “ActionCLIP: A New Paradigm for Video Action Recognition,” *arXiv.org*, 2021. https://arxiv.org/abs/2109.08472
[23] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” *arXiv.org*, 2014. https://arxiv.org/abs/1406.2199
[24] C. Feichtenhofer, A. Pinz, and A. Zisserman, “Convolutional Two-Stream Network Fusion for Video Action Recognition,” *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2016, doi: https://doi.org/10.1109/cvpr.2016.213.
[25] Y. Zhu, Z. Lan, S. Newsam, and A. G. Hauptmann, “Hidden Two-Stream Convolutional Networks for Action Recognition,” *arXiv.org*, 2017. https://arxiv.org/abs/1704.00389.
[26] Y. Xu, J. Yang, K. Mao, J. Yin, and S. See, “Exploiting inter-frame regional correlation for efficient action recognition,” *Expert Systems with Applications*, vol. 178, p. 114829, Sep. 2021, doi: https://doi.org/10.1016/j.eswa.2021.114829.
[27] H. Kataoka, T. Wakamiya, K. Hara, and Y. Satoh, “Would Mega-scale Datasets Further Enhance Spatiotemporal 3D CNNs?,” *arXiv.org*, 2020. https://arxiv.org/abs/2004.04968.
[28] C. Li, Q. Zhong, D. Xie, and S. Pu, “Collaborative Spatiotemporal Feature Learning for Video Action Recognition,” *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2019, doi: https://doi.org/10.1109/cvpr.2019.00806.
[29] Z. Qiu, T. Yao, and T. Mei, “Learning Spatio-Temporal Representation with Pseudo-3D Residual Networks,” *arXiv:1711.10305 [cs]*, Nov. 2017, Available: https://arxiv.org/abs/1711.10305
[30] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri, “Learning Spatiotemporal Features with 3D Convolutional Networks,” *arXiv.org*, 2014. https://arxiv.org/abs/1412.0767
[31] S. Song and J. Xiao, “Deep Sliding Shapes for Amodal 3D Object Detection in RGB-D Images,” Jun. 2016, doi: https://doi.org/10.1109/cvpr.2016.94.
[32] G. Varol, I. Laptev, and C. Schmid, “Long-Term Temporal Convolutions for Action Recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 6, pp. 1510–1517, Jun. 2018, doi: https://doi.org/10.1109/tpami.2017.2712608.
[33] Z. Liu, H. Tang, Y. Lin, and S. Han, “Point-Voxel CNN for Efficient 3D Deep Learning,” *arXiv.org*, Dec. 09, 2019. https://arxiv.org/abs/1907.03739
[34] Y. Li, S. Pirk, H. Su, C. R. Qi, and Guibas, Leonidas J, “FPNN: Field Probing Neural Networks for 3D Data,” *arXiv.org*, 2016. https://arxiv.org/abs/1605.06240.
[35] K. Hara, H. Kataoka, and Y. Satoh, “Can Spatiotemporal 3D CNNs Retrace the History of 2D CNNs and ImageNet?,” *IEEE Xplore*, Jun. 01, 2018. https://ieeexplore.ieee.org/document/8578783.
[36] J. Ahn, Y. Jang, and Joon Son Chung, “Slowfast Network for Continuous Sign Language Recognition,” Apr. 2024, doi: https://doi.org/10.1109/icassp48485.2024.10445841.
[37] C.-Y. Wu, R. Girshick, K. He, C. Feichtenhofer, and P. Krähenbühl, “A Multigrid Method for Efficiently Training Video Models,” *arXiv.org*, 2019. https://arxiv.org/abs/1912.00998.
[38] F. Xiao, Y. J. Lee, K. Grauman, J. Malik, and C. Feichtenhofer, “Audiovisual SlowFast Networks for Video Recognition,” *arXiv:2001.08740 [cs]*, Mar. 2020, [Online]. Available: https://arxiv.org/abs/2001.08740
[39] D. Chen, P. Wu, M. Chen, M. Wu, T. Zhang, and C. Li, “LS-VIT: Vision Transformer for action recognition

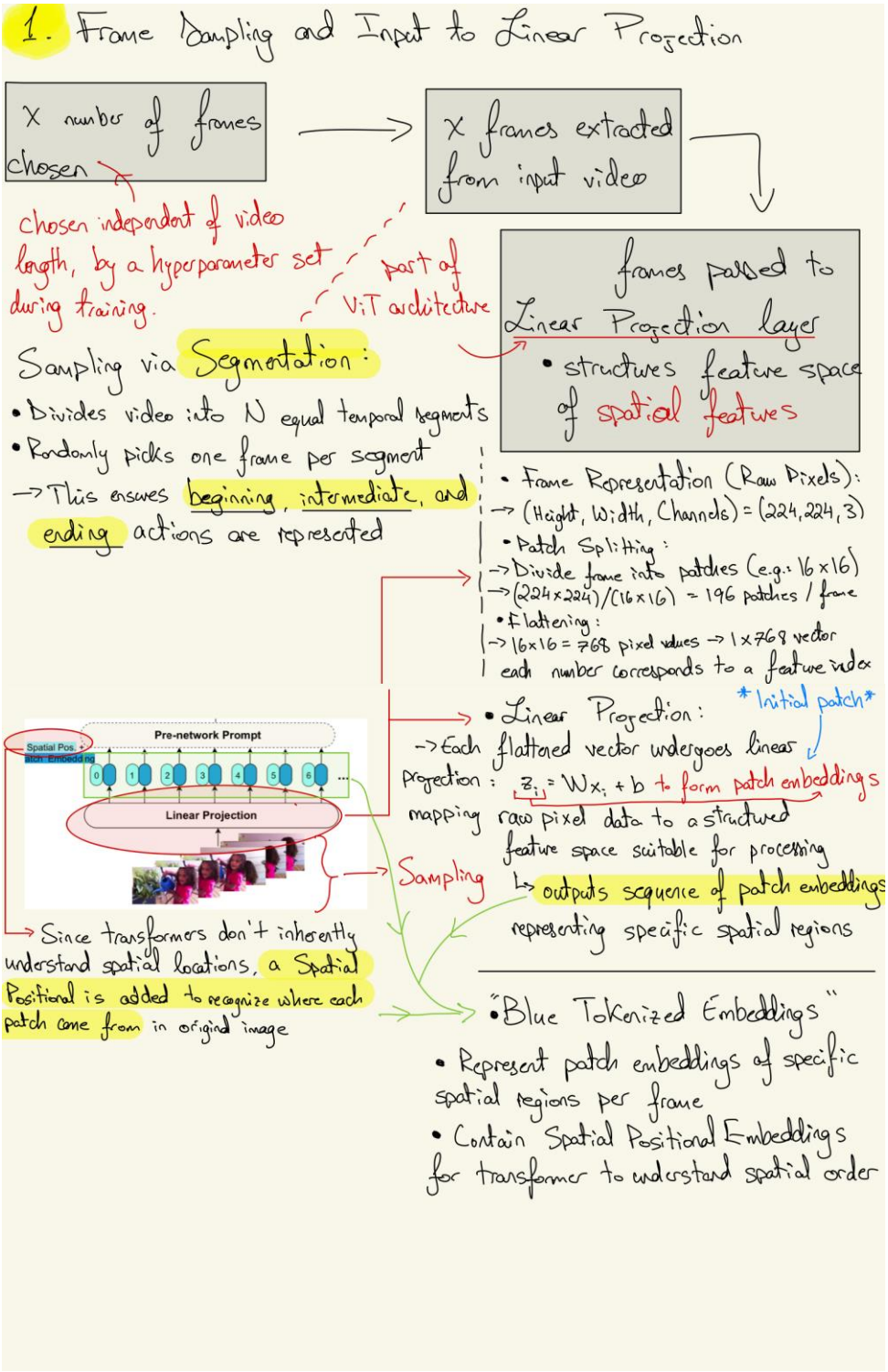
based on long and short-term temporal difference," *Frontiers in Neurorobotics*, vol. 18, Oct. 2024, doi: https://doi.org/10.3389/fnbot.2024.1457843.
[40] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked Autoencoders Are Scalable Vision Learners," *arXiv:2111.06377 [cs]*, Dec. 2021, Available: https://arxiv.org/abs/2111.06377
[41] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *arXiv:2010.11929 [cs]*, Oct. 2020, Available: https://arxiv.org/abs/2010.11929
[42] C.-C. Lin, K. Lin, L. Li, L. Wang, and Z. Liu, "Cross-modal Representation Learning for Zero-shot Action Recognition," *arXiv.org*, 2022. https://arxiv.org/abs/2205.01657.
[43] H. Xu *et al.*, "VideoCLIP: Contrastive Pre-training for Zero-shot Video-Text Understanding," *arXiv.org*, 2021. https://arxiv.org/abs/2109.14084.
[44] A. Radford *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," *arXiv:2103.00020 [cs]*, Feb. 2021, Available: https://arxiv.org/abs/2103.00020
[45] Y. Zhi, Z. Tong, L. Wang, and G. Wu, "MGSampler: An Explainable Sampling Strategy for Video Action Recognition," *arXiv.org*, 2021. https://arxiv.org/abs/2104.09952.
[46] A. Arnab, M. Dehghani, G. Heigold, C. Sun, M. Lučić, and C. Schmid, "ViViT: A Video Vision Transformer," *arXiv:2103.15691 [cs]*, Nov. 2021, Available: https://arxiv.org/abs/2103.15691
[47] "Hierarchical compositional representations for few-shot action recognition," *Arxiv.org*, 2024. https://arxiv.org/html/2208.09424v3.
[48] B. Heo, S. Park, D. Han, and S. Yun, "Rotary Position Embedding for Vision Transformer," *arXiv.org*, 2024. https://arxiv.org/abs/2403.13298
[49] K. Zhou, J. Yang, C. C. Loy, and Z. Liu, "Learning to Prompt for Vision-Language Models," *arXiv:2109.01134 [cs]*, Feb. 2022, Available: https://arxiv.org/abs/2109.01134
[50] D. Neimark, O. Bar, M. Zohar, and D. Asselmann, "Video Transformer Network," *arXiv.org*, 2021. https://arxiv.org/abs/2102.00719
[51] A. Kazemnejad, I. Padhi, K. N. Ramamurthy, P. Das, and S. Reddy, "The Impact of Positional Encoding on Length Generalization in Transformers," *arXiv.org*, Nov. 06, 2023. https://arxiv.org/abs/2305.19466
[52] "The Importance of Positional Encoding Initialization in Transformers for Relational Reasoning," *Arxiv.org*, 2023. https://arxiv.org/html/2406.08272v1.
[53] A. Vaswani *et al.*, "Attention Is All You Need," *arXiv*, Jun. 12, 2017. https://arxiv.org/abs/1706.03762
[54] X. Chu *et al.*, "Conditional Positional Encodings for Vision Transformers," *arXiv:2102.10882 [cs]*, Mar. 2021, Available: https://arxiv.org/abs/2102.10882
[55] E. Ku, "Transformer Architecture (Part 1 — Positional Encoding)," *Medium*, Aug. 21, 2023. https://medium.com/%40eugeneku123/transformer-architecture-part-1-positional-encoding-9b69c73140f7.
[56] J. Lin, C. Gan, and S. Han, "TSM: Temporal Shift Module for Efficient Video Understanding," *arXiv.org*, 2018. https://arxiv.org/abs/1811.08383
[57] "Papers with Code - Kinetics Dataset," *Paperswithcode.com*, 2022. https://paperswithcode.com/dataset/kinetics.
[58] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh, "Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks," *arXiv:1810.00825 [cs, stat]*, May 2019, Available: https://arxiv.org/abs/1810.00825
[59] M.-T. Luong, H. Pham, and C. D. Manning, "Effective Approaches to Attention-based Neural Machine Translation," *arXiv.org*, 2015. https://arxiv.org/abs/1508.04025
[60] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," *arXiv:2005.12872 [cs]*, May 2020, Available: https://arxiv.org/abs/2005.12872
[61] Y.-L. Chang, Z. Y. Liu, K.-Y. Lee, and W. Hsu, "Learnable Gated Temporal Shift Module for Deep Video Inpainting," *arXiv.org*, 2019. https://arxiv.org/abs/1907.01131.

[62] R. Hashiguchi and T. Tamaki, “Vision Transformer with Cross-attention by Temporal Shift for Efficient Action Recognition,” *arXiv.org*, 2022. https://arxiv.org/abs/2204.00452.
[63] Z. Zeng *et al.*, “Learning Transferable Spatiotemporal Representations from Natural Script Knowledge,” *arXiv.org*, 2022. https://arxiv.org/abs/2209.15280.
[64] J. Zhang, G. Wu, X. Bi, and Y. Cui, “Video Summarization Generation Network Based on Dynamic Graph Contrastive Learning and Feature Fusion,” *Electronics*, vol. 13, no. 11, p. 2039, May 2024, doi: https://doi.org/10.3390/electronics13112039.
[65] X. Teng *et al.*, “A Hierarchical Spatial–Temporal Cross-Attention Scheme for Video Summarization Using Contrastive Learning,” *Sensors (Basel)*, vol. 22, no. 21, pp. 8275–8275, Oct. 2022, doi: https://doi.org/10.3390/s22218275.
[66] N. Aziere and S. Todorovic, “Multistage temporal convolution transformer for action segmentation,” *Image and Vision Computing*, p. 104567, Oct. 2022, doi: https://doi.org/10.1016/j.imavis.2022.104567.
[67] D. Ahn, S. Kim, H. Hong, and B. C. Ko, “STAR-Transformer: A Spatio-temporal Cross Attention Transformer for Human Action Recognition,” *arXiv.org*, 2022. https://arxiv.org/abs/2210.07503.
[68] M. Khan and Yousef Hossni, “A comparative analysis of LSTM models aided with attention and squeeze and excitation blocks for activity recognition,” *Scientific Reports*, vol. 15, no. 1, Jan. 2025, doi: https://doi.org/10.1038/s41598-025-88378-6.
[69] L. Chen, R. Liu, X. Yang, D. Zhou, Q. Zhang, and X. Wei, “STTG-net: a Spatio-temporal network for human motion prediction based on transformer and graph convolution network,” *Visual Computing for Industry Biomedicine and Art*, vol. 5, no. 1, Jul. 2022, doi: https://doi.org/10.1186/s42492-022-00112-5.
[70] M. Nawhal and G. Mori, “Activity Graph Transformer for Temporal Action Localization,” *arXiv.org*, 2021. https://arxiv.org/abs/2101.08540.
[71] A. Piergiovanni and M. Ryoo, “Learning Multimodal Representations for Unseen Activities.” [Online]. Available: https://openaccess.thecvf.com/content_WACV_2020/papers/Piergiovanni_Learning_Multimodal_Representations_for_Unseen_Activities_WACV_2020_paper.pdf
[72] Z. Zheng, G. An, S. Cao, Z. Yang, and Q. Ruan, “PromptLearner-CLIP: Contrastive Multi-Modal Action Representation Learning with Context Optimization,” *Lecture notes in computer science*, pp. 554–570, Jan. 2023, doi: https://doi.org/10.1007/978-3-031-26316-3_33.
[73] Y. Rao *et al.*, “DenseCLIP: Language-Guided Dense Prediction with Context-Aware Prompting,” *arXiv.org*, 2021. https://arxiv.org/abs/2112.01518.
[74] Y. Yao *et al.*, “Prompt Tuning for Discriminative Pre-trained Language Models,” *Findings of the Association for Computational Linguistics: ACL 2022*, Jan. 2022, doi: https://doi.org/10.18653/v1/2022.findings-acl.273.
[75] X. Zhang, T. Wang, X. Li, H. Yang, and L. Yin, “Weakly-Supervised Text-driven Contrastive Learning for Facial Behavior Understanding,” *arXiv.org*, 2023. https://arxiv.org/abs/2304.00058.
[76] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” *arXiv:1503.02531 [cs, stat]*, Mar. 2015, Available: https://arxiv.org/abs/1503.02531
[77] Z. Weng, X. Yang, A. Li, Z. Wu, and Y.-G. Jiang, “Open-VCLIP: Transforming CLIP to an Open-vocabulary Video Model via Interpolated Weight Optimization,” *arXiv.org*, 2023. https://arxiv.org/abs/2302.00624.
[78] Z. Wu *et al.*, “Building an Open-Vocabulary Video CLIP Model with Better Architectures, Optimization and Data,” *arXiv.org*, 2023. https://arxiv.org/abs/2310.05010.
[79] A. Bulat, E. Sanchez, B. Martinez, and Georgios Tzimiropoulos, “Unsupervised open-vocabulary action recognition with an autoregressive model,” *OpenReview*, 2023. https://openreview.net/forum?id=IryGDUHxDE.
[80] D. Chatterjee, F. Sener, S. Ma, and A. Yao, “Opening the Vocabulary of Egocentric Actions,” *arXiv.org*, 2023. https://arxiv.org/abs/2308.11488.

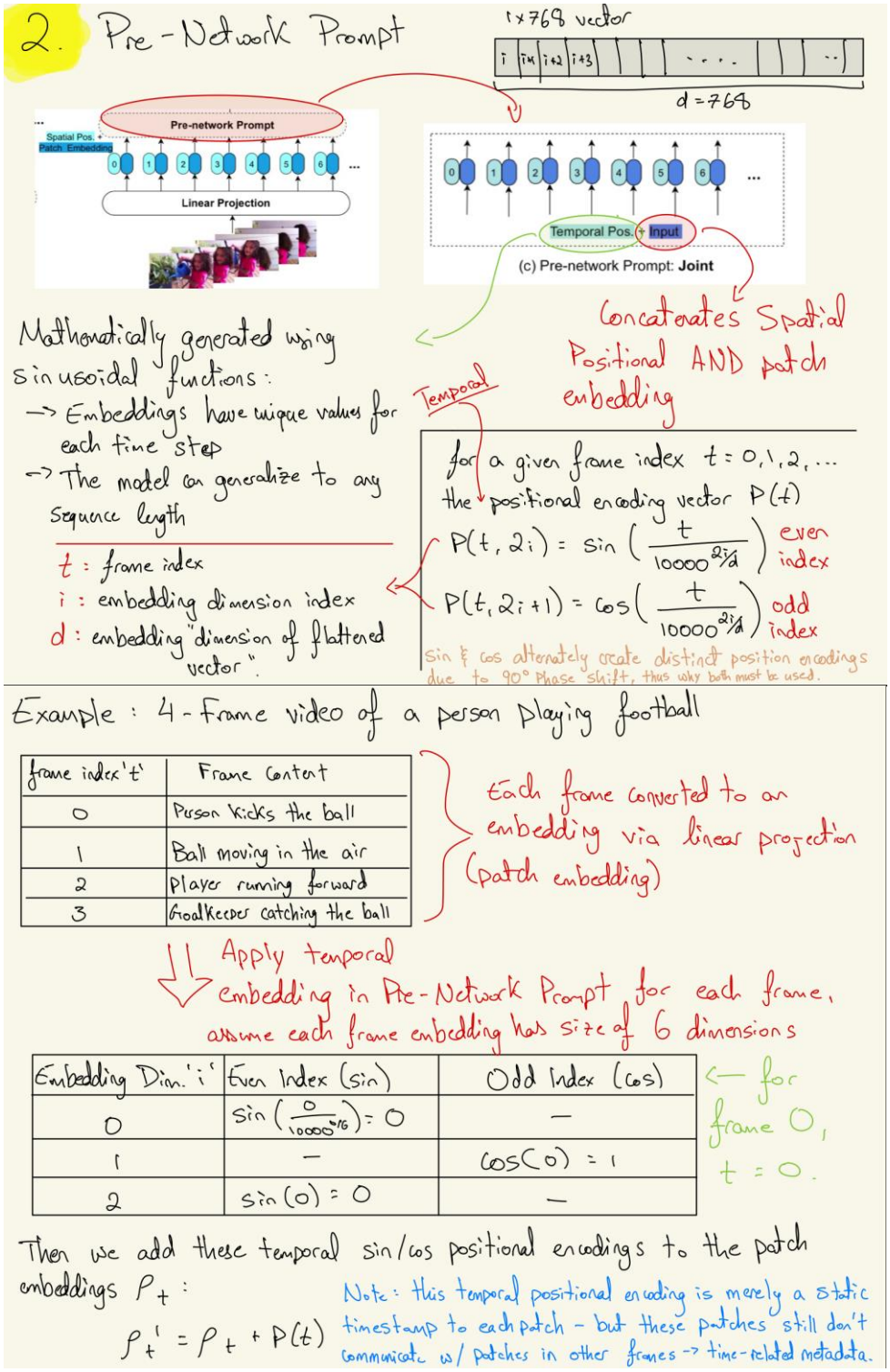
[81] W. Tu, W. Deng, and T. Gedeon, “A Closer Look at the Robustness of Contrastive Language-Image Pre-Training (CLIP),” *arXiv.org*, 2024. https://arxiv.org/abs/2402.07410.
[82] M. Ramanathan, W.-Y. Yau, and E. K. Teoh, “Human Action Recognition With Video Data: Research and Evaluation Challenges,” *IEEE Transactions on Human-Machine Systems*, vol. 44, no. 5, pp. 650–663, Oct. 2014, doi: https://doi.org/10.1109/thms.2014.2325871.
[83] “Gradient-based learning applied to document recognition - IEEE Journals & Magazine,” *Ieee.org*, 2019. https://ieeexplore.ieee.org/document/726791
[84] C. Sun, A. Myers, C. Vondrick, K. Murphy, and C. Schmid, “VideoBERT: A Joint Model for Video and Language Representation Learning,” *arxiv.org*, Apr. 2019, Available: https://arxiv.org/abs/1904.01766
[85] Z. Liu *et al.*, “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” *arXiv:2103.14030 [cs]*, Mar. 2021, Available: https://arxiv.org/abs/2103.14030
[86] M. Wang, J. Xing, J. Mei, Y. Liu, and Y. Jiang, “ActionCLIP: Adapting Language-Image Pretrained Models for Video Action Recognition,” *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–13, Nov. 2023, doi: https://doi.org/10.1109/tnnls.2023.3331841.
[87] Valter Estevam, Rayson Laroca, H. Pedrini, and D. Menotti, “Tell me what you see: A zero-shot action recognition method based on natural language descriptions,” *Multimedia tools and applications*, Sep. 2023, doi: https://doi.org/10.1007/s11042-023-16566-5.
[88] K. Soomro, Zamir, Amir Roshan, and M. Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in The Wild,” *arXiv.org*, 2012. https://arxiv.org/abs/1212.0402
[89] W. Sultani, C. Chen, and M. Shah, “Real-world Anomaly Detection in Surveillance Videos,” *arXiv.org*, 2018. https://arxiv.org/abs/1801.04264
[90] M. Jansen, “UCF101 - Action Recognition,” *Kaggle.com*, 2023. https://www.kaggle.com/datasets/matthewjansen/ucf101-action-recognition
[91] “UCF Crime Dataset,” *www.kaggle.com*. https://www.kaggle.com/datasets/odins0n/ucf-crime-dataset
[92] sallymmx, “ActionCLIP/README.md at master · sallymmx/ActionCLIP,” *GitHub*, 2021. https://github.com/sallymmx/ActionCLIP/blob/master/README.md.
[93] H. He and E. A. Garcia, “Learning from Imbalanced Data - IEEE Journals & Magazine,” *Ieee.org*, 2009. https://ieeexplore.ieee.org/document/5128907
[94] K. Zhou, J. Yang, C. C. Loy, and Z. Liu, “Conditional Prompt Learning for Vision-Language Models,” *arXiv.org*, Oct. 06, 2022. https://arxiv.org/abs/2203.05557
[95] A. Prasad, P. Hase, X. Zhou, and M. Bansal, “GrIPS: Gradient-free, Edit-based Instruction Search for Prompting Large Language Models,” *arXiv.org*, Mar. 14, 2022. https://arxiv.org/abs/2203.07281
[96] J. Cao, H. Wu, and K. Kitani, “Track Targets by Dense Spatio-Temporal Position Encoding,” *arXiv.org*, 2022. https://arxiv.org/abs/2210.09455.
[97] Y. Zhao, K. L. Man, J. Smith, and S.-U. Guan, “A novel two-stream structure for video anomaly detection in smart city management,” *The Journal of Supercomputing*, vol. 78, no. 3, pp. 3940–3954, Aug. 2021, doi: https://doi.org/10.1007/s11227-021-04007-9.

Appendix A. Informative Notes

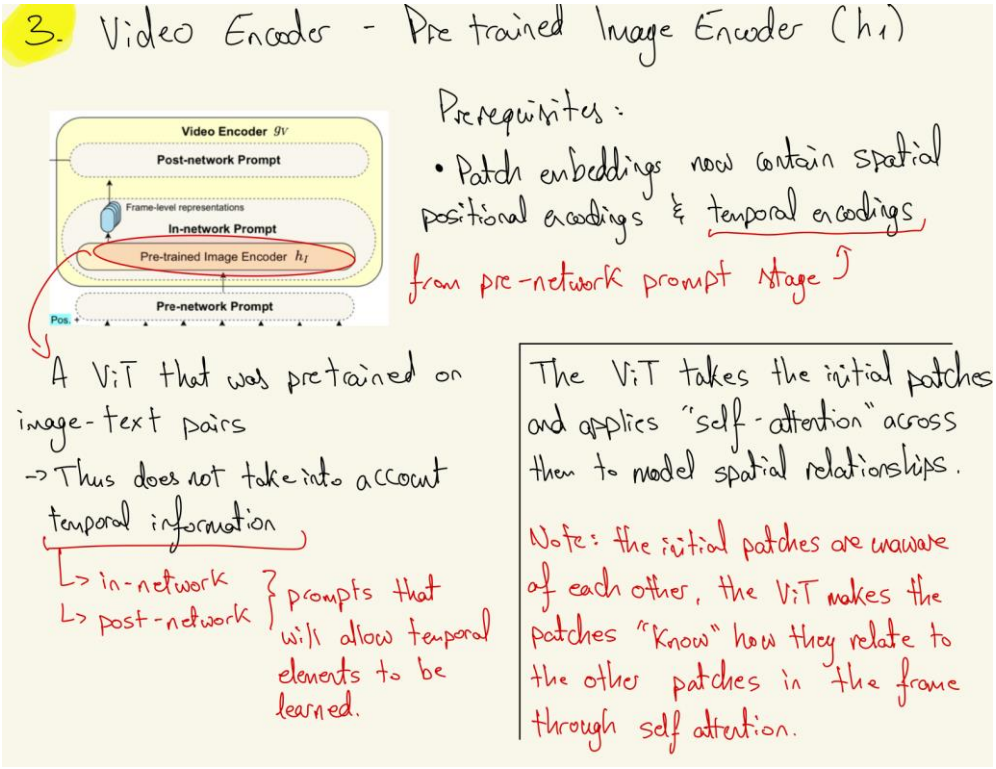
A.1 Frame Sampling

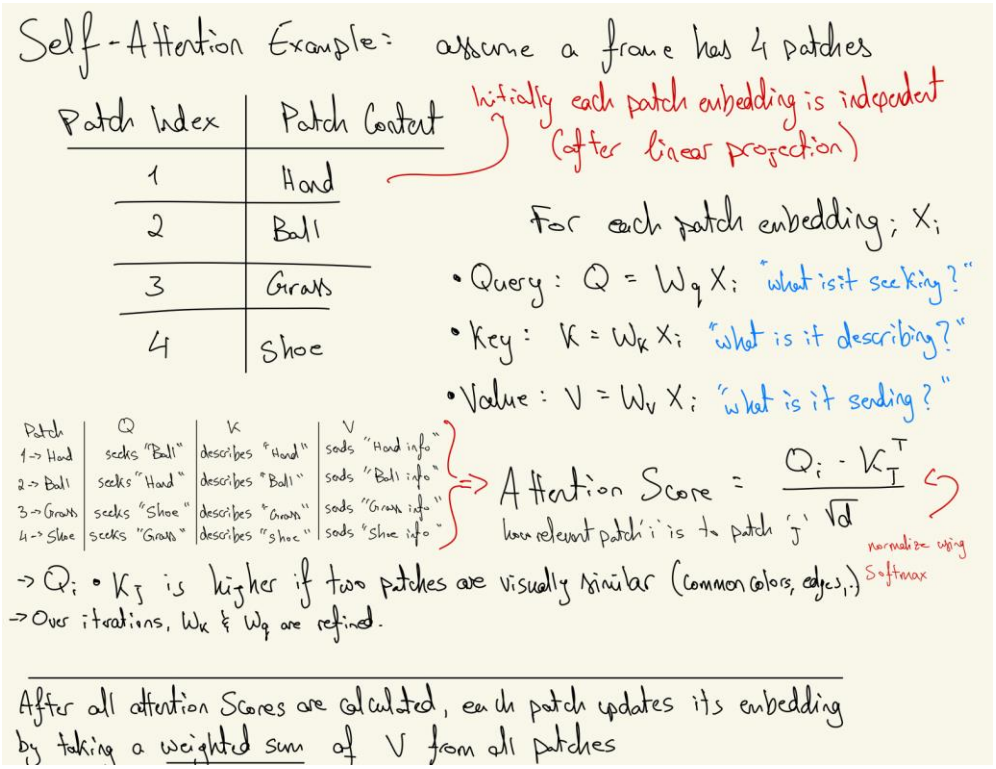


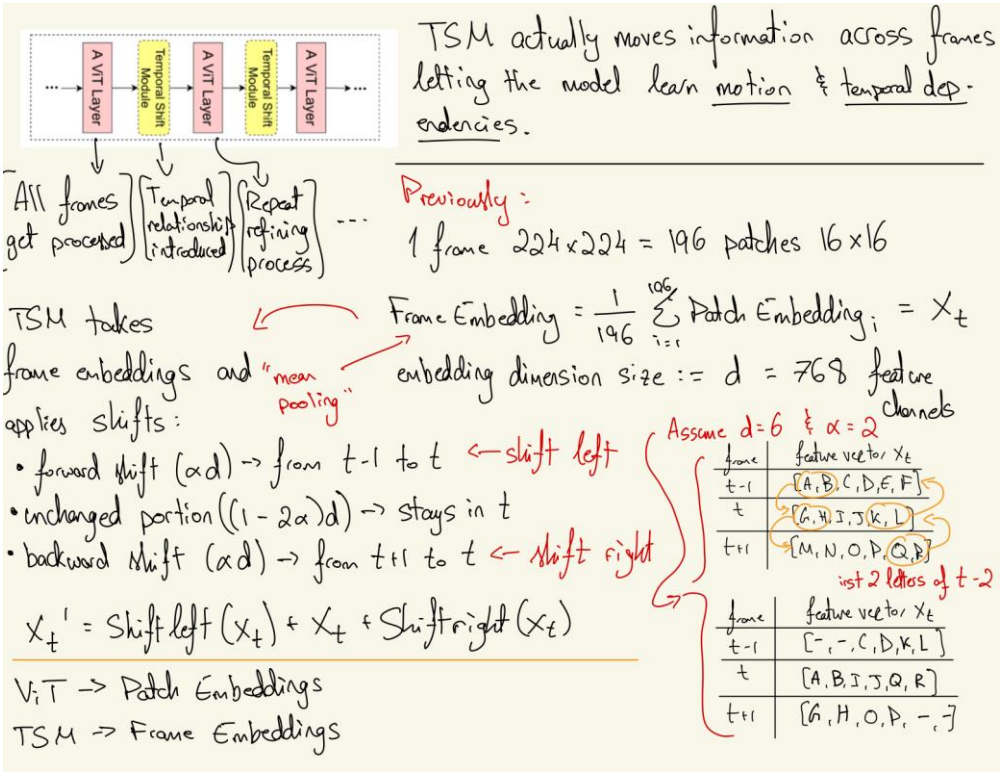
A.2 Pre-Network Prompt



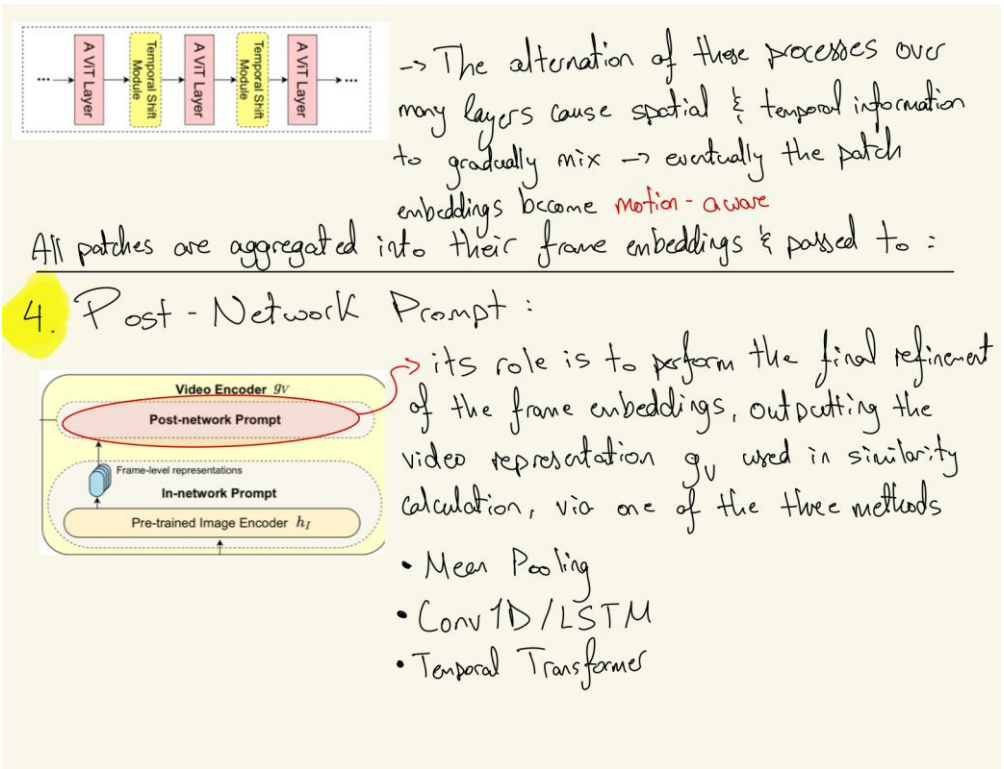
A.3 In-Network Prompt







A.4 Post-Network Prompt



Mean Pooling (Mean P):

$$V = \frac{1}{T} \sum_{t=1}^{T} F_t$$

V → video embedding
T → # of frames
F_t → frame embedding at t

→ Computationally efficient
→ Ignores order of frames
→ Does not model long-range dependencies between frames
→ Essentially, motion-sensitive information that was present in the temporal encodings of the frame embeddings get averaged out, reducing its effectiveness and overlooking the complexity of prior steps.

Conv 1D

$$V_t = \sum_{i=0}^{K-1} W_i \cdot F_{t+i}$$

As seen in the equation, Conv 1D works well for modelling information in a relatively close time frame (depending on K, kernel size). Thus when adjacent frames relay information sequentially, it is easy for Conv 1D to recognize an action. If an initial frame contains important information that relates to a distant frame, it would be challenging to recognize the action.

A more preferable method is LSTM (long Short-term Memory)

LSTM:

→ Forget Gate: how much of past info to keep? - f_t
→ Input Gate: how much of new info to keep? - i_t

$$f_t = \sigma(W_f \cdot [F_t, h_{t-1}] + b_f) \qquad i_t = \sigma(W_i [F_t, h_{t-1}] + b_i)$$

h_t: hidden state

Candidate Memory Update → $\tilde{C}_t = \tanh(W_c \cdot [F_t, h_{t-1}] + b_c)$

Update Cell State → $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$

→ Output Gate: how much of the new state to pass forward? - o_t

$$o_t = \sigma(W_o \cdot [F_t, h_{t-1}] + b_o)$$

$h_t = o_t \odot \tanh(C_t)$ → final hidden state passed to next state.

The mechanism allows it to track motion along long time spans

A.5 Preamble Notes

MLP

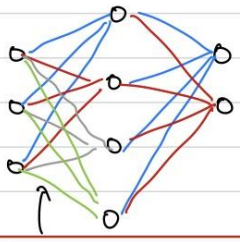
$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) \quad (5.1)$$

x_i : input values
w_i : weights
b : bias term
f : activation fn
y : output

This is a single perceptron

Multilayer Perceptron:

input layer	hidden layer(s)	output layer
receive raw data (img, signal, etc.)	captures complex relation-ships in the data	provides prediction (classification) of the result



each neuron in a layer is connected to each neuron in the subsequent layer

Each connection has its own weight, & each neuron applies an activation fn to its weighted sum

Activation Fns:
introduce non-linearity into the network. Why?

-> Sigmoid: maps input to a value between 0 & 1. IDEAL FOR BINARY CLASSIFICATIONS.

-> ReLU: sets all -ve values to 0 & leaves all +ve unchanged. Efficient for deep networks since it avoids the vanishing gradient problem. ?

-> Softmax: used in output layer for multi-class classification, turning the outputs into probabilities that sum to 1.

Forward Propagation: process of passing input data through each layer to obtain an output. In each hidden (h) layer, we compute the weighted sum of the inputs, apply the activation fn, and pass the result to the next layer.

MLP

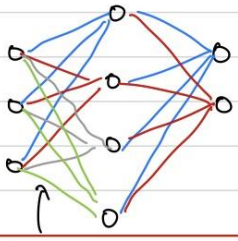
$$y = f\left(\sum_{i=1} w_i x_i + b\right) \quad (5.1)$$

x_i : input values
w_i : weights
b : bias term
f : activation fn
y : output

This is a single perceptron

Multilayer Perceptron:

input layer	hidden layer(s)	output layer
receive raw data (img, signal, etc.)	captures complex relation-ships in the data	provides prediction (classification) of the result



each neuron in a layer is connected to each neuron in the subsequent layer

Each connection has its own weight, & each neuron applies an activation fn to its weighted sum

Activation Fns:

introduce non-linearity into the network. Why?

-> Sigmoid: maps input to a value between 0 & 1. IDEAL FOR BINARY CLASSIFICATIONS.

-> ReLU: sets all -ve values to 0 & leaves all +ve unchanged. Efficient for deep networks since it avoids the vanishing gradient problem. ?

-> Softmax: used in output layer for multi-class classification, turning the outputs into probabilities that sum to 1.

Forward Propagation: process of passing input data through each layer to obtain an output. In each hidden (h) layer, we compute the weighted sum of the inputs, apply the activation fn, and pass the result to the next layer.

Loss Fn: measures the error between the network's prediction & the actual values.

-> Mean Squared Error (MSE): used in regression tasks to penalize large errors

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

actual <-> predicted

By adjusting the weights, this margin can be minimized.

by doing this

Backpropagation & Gradient Descent:

1: Compute the loss - after forward propagation, calculate error using loss fn
2: Calculate gradients - for each weight, compute the gradient of the loss w/ respect to that weight (shows how much the loss would change if the weight was adjusted)
3: Update weights - using gradient descent, adjust each weight in the opposite direction of the gradient to minimize loss. Use learning rate (α) to control the size of the w adjustment

Gradient Descent Weight Update Rule:

$$w := w - \alpha \frac{\partial Loss}{\partial w}$$

TRAINING THE MLP:

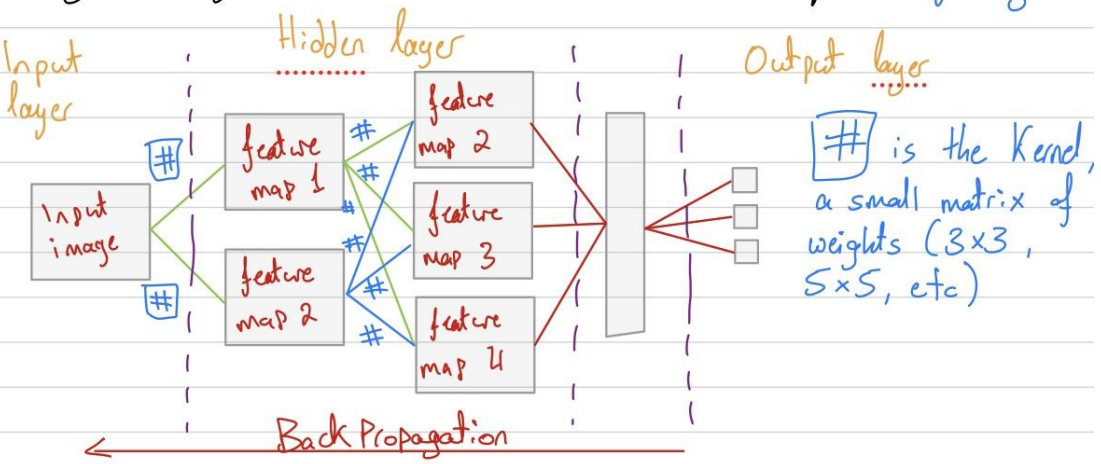
Stops once the loss converges (i.e., stops decreasing significantly)

CNN

CNNs are particularly designed to detect spatial patterns, ideal for processing grid-like data like 2D images.

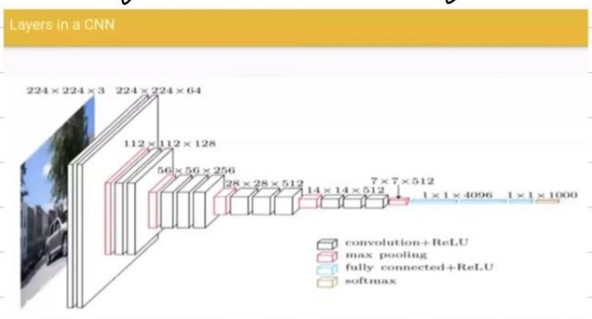
Why not use MLP for images?

High resolution images would require large amount of parameters, leading to high computational costs and risk of overfitting.



From the input image, the Kernel generates a new set of images, from which a new Kernel creates another new set of images called feature maps.

The weights are the values generated by the Kernel.



The input image eventually gets condensed to a 1x1, 1 dimensional image.

It does so by Parallel Computing

Every Kernel helps identify a different feature of the image.

Key difference to MLP:
CNNs will store (keep in mind) what all pixels are relative to each other's space. Hence, spatial awareness. Using ReLU & max pooling

I: img patch, K: Kernel

Convolutional Layer: $(I * K)_{ij} = \sum_m \sum_n I_{(i+m)(j+n)} K_{mn}$

Allows the network to focus on a specific part of an image, multiplying Kernel values w/ the pixel values it overlaps and summing the result.
- Kernel values: weights of the filter
- pixel values: could be brightness, color intensity, for greyscale its a single value, for RGB it's 3

Stride:
The number of pixels the Kernel moves after each step. If stride = 1, Kernel moves 1 pixel at a time. Direction: horizontally & vertically

Padding:
Padding adds extra pixels around the edges of the input to ensure the output has same dimensions. Otherwise every convolution would shrink the dimensions, and more challenging to detect features at edges.

Pooling: A reduction mechanism to reduce height × width of the feature map (thus fewer parameters)
→ Max Pooling: max value of what?
→ Avg Pooling:

What direction does the Stride shift the Kernel across?
Why might we choose Avg pooling instead of Max pooling?

Summary

we are in layer 'n'

→ Layer n-1 undergoes convolution, where it applies a series of filters (kernels) to different parts of an image resulting in a feature map (matrix) containing that part's feature values.

→ An activation function is applied to the map to add non-linearity hence all -ve values = 0 (if using ReLU), thus only important features (+ve) are passed forward.

→ Now the +ve maps enter the Max Pooling layer, here each 2x2 region of the map is scanned & only the highest value in each region is selected to represent that area of the map (typically the highest value is the most prominent feature that holds the most information).

Transformers

Transformers are typically us in Natural Language Processing cases (NLP):

- Self-Attention Mechanism
- Feedforward Network

Self-Attention Mechanism:

→ Input Embedding - first, input is transformed to a vector called an Embedding, representing each word in a sequence

→ Attention Scores - scores calculated to represent how each word in the input relates to every other part using queries ($\underline{Q}$), keys ($\underline{K}$), & values ($\underline{V}$).

Attention Score = norm ($\underline{Q} \cdot \underline{K}$), normalized using softmax

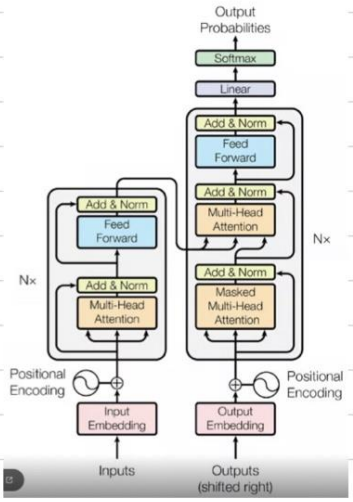
After applying Self-Attention, the output passes through a feedforwarding network layer, similar to MLP, which adds further transformations & Non-Linearity.

Positional Encoding:

Since they do not have inherent order, the position of each element of the input must be encoded.

→ Encoder: Processes input sequence via self-Attention & feedforwarding

→ Decoder: Processes encoded info & produces output sequence. Contains Encoder-Decoder Attention



Optimization:

- Multi-headed Self-Attention: instead of calculating one attation pattern?, it computes several in parallel (each w/ diff. learned parameters) where each 'head' focuses on diff. aspects of the input, allowing the model to capture a range of relationships.

How does the Embedding transform a sentence of strings into a vector representation of each word?

- First the sentence is split into a smaller unit (words or sub-words), for which each unit is assigned a token (tokenization) and each token is mapped to an integer index from a vocabulary.
- Second, the integer indices are used to retrieve initialized vectors from an embedding lookup table (like a dictionary).
- Words w/ similar meaning like 'King' and 'queen' are closer in this vector space. The vector embeddings now allow the model to work w/ text data in numerical form whilst preserving semantic & positional relationships.

Roles of Queries (Q), Keys (K), & Values (V)?

For each token w/ Embedding x_i, $i = 1, 2, \dots, n$. $n :=$ # of tokens in an input

There exists weights W_{Q_i}, W_{K_i}, W_{V_i} which have been computed for each token during training.

$$Q = x_i W_{Q_i} \quad , \quad K = x_i W_{K_i} \quad , \quad V = x_i W_{V_i}$$

To determine how much 'focus' one token should have on another:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V$$

Step-by-Step Example:

Encoder:

Input = "I like apples"

→ Tokenization and Embedding

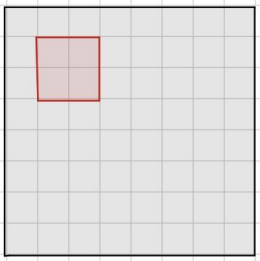
Results in 3 token embeddings for "I", "like", and "apples".

→ Positional Encoding:

Positional information is added to each respective embedding to distinguish the order of words.

→ Self-Attention:

In each encoder layer, attention scores are calculated for each token relative to one another. "I" might pay more attention to "like" than "apples". This adds contextual information to each embedding.



Red: Kernel → $2 \times (5 \times 5 \times 1)$ with a unique bias term and iterate over the whole image

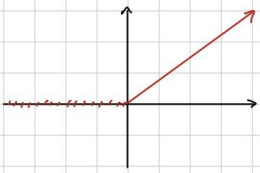
Using: $a_i = ReLu(z_i)$, $z_i = \sum_{i=1}^{K} w_i \times a_i + b$ (multiply each w in the kernel by the overlapping values from the input)

w/ Stride = 1, we shift the Kernel by 1 px at every step then 1 step down and back to the beginning (iteration)

we get a matrix of same dimension of kernel with sum of all products, then add bias term to get z_{183} (183 iterations)

After 183 iterations, we pass z_{183} to the ReLU function.

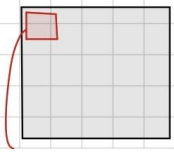
$$ReLU(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$



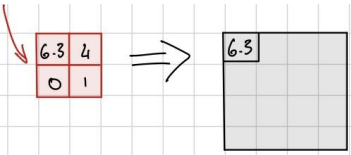
Then we do the exact same thing for the 2nd Kernel and move on to the pooling layer. (We now have 2 matrices)

These 2 matrices / images are called Channels

Max Pooling, pool size (2×2), stride = 2:



Red: Pooling filter, simply extracts the largest value and places it into a new channel



This new channel is ½ the dimension size of the previous channel.

Again, there are 2 channels since we had 2 original kernels.

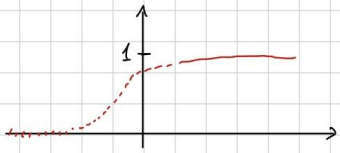
Convolution 2.0

We have 4 filters, but 2 channels so we actually have 8 filters:

$$a_i = s(z_i), \quad z_i = \sum_{j=1}^{c} \sum_{i=r}^{x} w_i(j) \times a_i(j) + b$$

→ Combines 2 filters together

$$s(x) = \frac{1}{1+e^{-x}}$$



Sigmoid outputs values between 0 & 1.

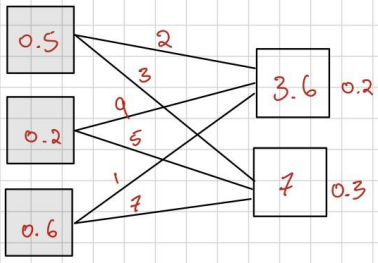
Thus, we have 4 new channels with matrix entries ranging from 0 (black) to 1 (white)

Max Pooling 2.0

Repeat the same step as 1.0 to generate 4 filters w/ matrix extracting the most activated pixels, ½ dimension size

We then flatten out the 4 matrices. From 4×(5×5) matrices to a flat layer w/ 100 nodes, each node corresponding to a pixel

Output Layer (Fully connected)



Each output node has a bias term that will be added later in the process.

The input nodes are connected to the output nodes through some weights.

Multiply input node by the corresponding weight and place that product in the output node.

After all inputs have contributed to the output, we add the bias term to give the final result.

Computationally fast way of doing that is by matrix multiplication:

$$z = XW + b \Rightarrow z = [0.5 \quad 0.2 \quad 0.6] \times \begin{bmatrix} 2 & 3 \\ 9 & 5 \\ 1 & 7 \end{bmatrix} + [0.2 \quad 0.5]$$

$$z = [3.6 \quad 7]$$

Appendix B. Selected Listings

In this Appendix are the code listings used in the project. The GitHub repository in Reference [92] contains the entirety of ActionCLIP ready for implementation on GPU, but some of these files needed adjustments to perform on CPU. These files contain the term (Modified) in their headings, such as Appendix B.1. For custom training and testing, new files needed to be created and are listed in this Appendix.

B.1 environment.yml (Modified)

```
01: name: actionclip_compatible
02: channels:
03:   - pytorch
04:   - conda-forge
05:   - defaults
06:
07: dependencies:
08:   - python=3.8
09:   - pip
10:   - numpy=1.20
11:   - scipy
12:   - pandas
13:   - matplotlib
14:   - scikit-learn
15:   - tqdm
16:   - pillow
17:   - cython
18:   - ffmpeg=4.3
19:   - pytorch=1.8.1
20:   - torchvision=0.9.1
21:   - torchaudio=0.8.1
22:   - cudatoolkit=11.1
23:   - pyyaml
24:   - opencv
25:   - pip:
26:       - audioread
27:       - wandb
28:       - dotmap
29:       - pathlib
30:       - RandAugment
31:       - absl-py
32:       - addict
33:       - antlr4-python3-runtime
34:       - apex
35:       - black
36:       - click
37:       - configparser
38:       - decorator
39:       - dill
40:       - docker-pycreds
41:       - einops
42:       - fairseq
43:       - fire
44:       - ftfy
45:       - future
46:       - fvcore
47:       - gitpython
48:       - google-auth
49:       - google-auth-oauthlib
50:       - grpcio
```

```
51:       - hydra-core
52:       - iopath
53:       - imageio
54:       - imageio-ffmpeg
55:       - jpeg4py
56:       - lightseq
57:       - lmdb
58:       - markdown
59:       - mmcv-full
60:       - model-index
61:       - moviepy
62:       - multiprocess
63:       - ninja
64:       - oauthlib
65:       - omegaconf
66:       - openmim
67:       - parameterized
68:       - pathos
69:       - pydot
70:       - pytorch-lightning==1.3.8
71:       - pytorchvideo==0.1.2
72:       - regex
73:       - sacrebleu
74:       - sentry-sdk
75:       - simplejson
76:       - sklearn
77:       - tabulate
78:       - tensorboard
79:       - tensorboard-plugin-wit
80:       - tensorboardx
81:       - termcolor
82:       - toml
83:       - torchmetrics
84:       - umsgpack
85:       - wandb
86:       - wcwidth
87:       - werkzeug
88:       - yacs
89:       - yapf
90:       - yarl
```

B.2 frame_extract.py

```
01: import os
02: import cv2
03:
04: video_root = 'C:/Users/nabil/Downloads/Datasets/UCF/UCF101/UCF-101'
05: output_root = 'C:/Users/nabil/Downloads/Datasets/UCF101_frames'
06:
07: for class_name in os.listdir(video_root):
08:     class_path = os.path.join(video_root, class_name)
09:     if not os.path.isdir(class_path): continue
10:
11:     for video_file in os.listdir(class_path):
12:         if not video_file.endswith('.avi'):
13:             continue
14:
15:         video_path = os.path.join(class_path, video_file)
16:         video_name = os.path.splitext(video_file)[0]
17:         output_dir = os.path.join(output_root, class_name,
video_name)
18:
```

```
        os.makedirs(output_dir, exist_ok=True)

        cap = cv2.VideoCapture(video_path)
        frame_idx = 1

        while True:
            ret, frame = cap.read()
            if not ret:
                break
            frame_filename = os.path.join(output_dir,
f'img_{frame_idx:05d}.jpg')
            cv2.imwrite(frame_filename, frame)
            frame_idx += 1

        cap.release()
```

B.3 validation_split.py

```
import os
import random
import csv

label_csv = 'lists/ucf_labels.csv'
frame_root = 'C:/Users/nabil/Downloads/Datasets/UCF101_frames'
output_file = 'lists/ucf101/val_rgb_split1.txt'
val_ratio = 0.05  # 5% per class

os.makedirs(os.path.dirname(output_file), exist_ok=True)

with open(label_csv, 'r') as f:
    reader = csv.DictReader(f)
    label_dict = {row['name']: int(row['id']) for row in reader}

with open(output_file, 'w') as out_f:
    for cls in os.listdir(frame_root):
        cls_path = os.path.join(frame_root, cls)
        if not os.path.isdir(cls_path): continue

        videos = os.listdir(cls_path)
        random.shuffle(videos)
        val_count = max(1, int(len(videos) * val_ratio))

        for vid in videos[:val_count]:
            out_f.write(f"{cls}/{vid} 0 {label_dict[cls]}\n")
```

B.4 generate_crime.py

```
import os
import csv

dataset_path = "C:/Users/nabil/Downloads/Datasets/kaggle/Train"
output_train_list = "lists/ucf_crime/train_rgb.txt"
output_label_list = "lists/ucf_crime/crime_labels.csv"

class_labels = {
    "NormalVideos": 0,
    "Shoplifting": 1
}

with open(output_label_list, mode='w', newline='') as label_file:
    writer = csv.writer(label_file)
    for class_name, label in class_labels.items():
```

```
16:         writer.writerow([class_name, label])
17:
18: print(f"Label list saved to {output_label_list}")
19:
20: with open(output_train_list, mode='w') as train_file:
21:     for class_name, label in class_labels.items():
22:         class_folder = os.path.join(dataset_path, class_name)
23:         for filename in os.listdir(class_folder):
24:             if filename.endswith(".png"):
25:                 relative_path = f"{class_name}/{filename}"
26:                 train_file.write(f"{relative_path} {label}\n")
27:
28: print(f"Training list saved to {output_train_list}")
```

B.5 train.py (Modified)

```
001: # Code for "ActionCLIP: A New Paradigm for Action Recognition"
002: # arXiv:
003: # Mengmeng Wang, Jiazheng Xing, Yong Liu
004:
005: import os
006: import torch
007: import torch.nn as nn
008: from datasets import Action_DATASETS
009: from torch.utils.data import DataLoader
010: from tqdm import tqdm
011: import wandb
012: import clip
013: import argparse
014: import shutil
015: from pathlib import Path
016: import yaml
017: from dotmap import DotMap
018: import pprint
019: from modules.Visual_Prompt import visual_prompt
020: from utils.KLLoss import KLLoss
021: from test import validate
022: from utils.Augmentation import *
023: from utils.solver import _optimizer, _lr_scheduler
024: from utils.tools import *
025: from utils.Text_Prompt import *
026: from utils.saving import  *
027:
028: class TextCLIP(nn.Module):
029:     def __init__(self, model) :
030:         super(TextCLIP, self).__init__()
031:         self.model = model
032:
033:     def forward(self,text):
034:         return self.model.encode_text(text)
035:
036: class ImageCLIP(nn.Module):
037:     def __init__(self, model) :
038:         super(ImageCLIP, self).__init__()
039:         self.model = model
040:
041:     def forward(self,image):
042:         return self.model.encode_image(image)
043:
044: def main():
045:     global args, best_prec1
046:     global global_step
```

```
047:     parser = argparse.ArgumentParser()
048:     parser.add_argument('--config', '-cfg', default='')
049:     parser.add_argument('--log_time', default='')
050:     args = parser.parse_args()
051:     with open(args.config, 'r') as f:
052:         config = yaml.safe_load(f)
053:     working_dir = os.path.join('./exp', config['network']['type'],
config['network']['arch'], config['data']['dataset'], args.log_time)
054:
wandb.init(project=config['network']['type'],name='{}_{}_{}_{}'.format(
args.log_time,config['network']['type'], config['network']['arch'],
config['data']['dataset']))
055:     print('-' * 80)
056:     print(' ' * 20, "working dir: {}".format(working_dir))
057:     print('-' * 80)
058:
059:     print('-' * 80)
060:     print(' ' * 30, "Config")
061:     pp = pprint.PrettyPrinter(indent=4)
062:     pp.pprint(config)
063:     print('-' * 80)
064:
065:     config = DotMap(config, _dynamic=False)
066:
067:     Path(working_dir).mkdir(parents=True, exist_ok=True)
068:     shutil.copy(args.config, working_dir)
069:     shutil.copy('train.py', working_dir)
070:
071:     device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
072:
073:     model, clip_state_dict = clip.load(config.network.arch,
device=device, jit=False, tsm=config.network.tsm,
T=config.data.num_segments, dropout=config.network.drop_out,
emb_dropout=config.network.emb_dropout, pretrain=config.network.init,
joint=config.network.joint)
074:
075:     if device.type == "cpu":
076:         model.float()  #ensure model is in float32
077:
078:     transform_train = get_augmentation(True, config)
079:     transform_val = get_augmentation(False, config)
080:
081:     if config.data.randaug.N > 0:
082:         transform_train = randAugment(transform_train, config)
083:
084:     print('train transforms:
{}'.format(transform_train.transforms))
085:     print('val transforms: {}'.format(transform_val.transforms))
086:
087:     fusion_model = visual_prompt(config.network.sim_header,
clip_state_dict, config.data.num_segments)
088:     model_text = TextCLIP(model)
089:     model_image = ImageCLIP(model)
090:
091:     model_text = model_text.to(device)
092:     model_image = torch.nn.DataParallel(model_image).to(device)
093:     fusion_model = torch.nn.DataParallel(fusion_model).to(device)
094:
095:     if device.type == "cpu":
096:         model_text.float()
097:         model_image.module.float()
```

```
098:         fusion_model.module.float()
099:
100:     wandb.watch(model)
101:     wandb.watch(fusion_model)
102:
103:     train_data = Action_DATASETS(config.data.train_list,
config.data.label_list, num_segments=config.data.num_segments,
image_tmpl=config.data.image_tmpl,
random_shift=config.data.random_shift, transform=transform_train)
104:     train_loader = DataLoader(train_data,
batch_size=config.data.batch_size, num_workers=config.data.workers,
shuffle=True, pin_memory=False, drop_last=True)
105:     val_data = Action_DATASETS(config.data.val_list,
config.data.label_list, random_shift=False,
num_segments=config.data.num_segments,
image_tmpl=config.data.image_tmpl, transform=transform_val)
106:     val_loader = DataLoader(val_data,
batch_size=config.data.batch_size, num_workers=config.data.workers,
shuffle=False, pin_memory=False, drop_last=True)
107:
108:     loss_img = KLLoss()
109:     loss_txt = KLLoss()
110:
111:     start_epoch = config.solver.start_epoch
112:
113:     if config.pretrain and os.path.isfile(config.pretrain):
114:         print(f"=> loading checkpoint '{config.pretrain}'")
115:         checkpoint = torch.load(config.pretrain,
map_location=device)
116:         model.load_state_dict(checkpoint['model_state_dict'])
117:
fusion_model.load_state_dict(checkpoint['fusion_model_state_dict'])
118:         del checkpoint
119:     elif config.resume and os.path.isfile(config.resume):
120:         print(f"=> loading checkpoint '{config.resume}'")
121:         checkpoint = torch.load(config.resume,
map_location=device)
122:         model.load_state_dict(checkpoint['model_state_dict'])
123:
fusion_model.load_state_dict(checkpoint['fusion_model_state_dict'])
124:         start_epoch = checkpoint['epoch']
125:         del checkpoint
126:
127:     classes, num_text_aug, text_dict = text_prompt(train_data)
128:
129:     optimizer = _optimizer(config, model, fusion_model)
130:     lr_scheduler = _lr_scheduler(config, optimizer)
131:
132:     best_prec1 = 0.0
133:     if config.solver.evaluate:
134:         prec1 = validate(start_epoch, val_loader, classes, device,
model, fusion_model, config, num_text_aug)
135:         return
136:
137:     for epoch in range(start_epoch, config.solver.epochs):
138:         model_image.train()
139:         model_text.train()
140:         fusion_model.train()
141:         for kkk, (images, list_id) in
enumerate(tqdm(train_loader)):
142:             if config.solver.type != 'monitor' and ((kkk+1) == 1
or (kkk+1) % 10 == 0):
```

```
143:                 lr_scheduler.step(epoch + kkk / len(train_loader))
144:
145:             optimizer.zero_grad()
146:
147:             images = images.view((-1, config.data.num_segments, 3)
+ images.size()[-2:])
148:             b, t, c, h, w = images.size()
149:             text_id = numpy.random.randint(0, 2,
size=len(list_id))
150:             texts = torch.stack([text_dict[j][i, :] for i, j in
zip(list_id, text_id)])
151:
152:             images = images.to(device).view(-1, c, h, w)
153:             texts = texts.to(device)
154:
155:             image_embedding = model_image(images)
156:             image_embedding = image_embedding.view(b, t, -1)
157:             image_embedding = fusion_model(image_embedding)
158:
159:             text_embedding = model_text(texts)
160:
161:             if config.network.fix_text:
162:                 text_embedding = text_embedding.detach()
163:
164:             logit_scale = model.logit_scale.exp()
165:             logits_per_image, logits_per_text =
create_logits(image_embedding, text_embedding, logit_scale)
166:
167:             ground_truth = torch.tensor(gen_label(list_id),
dtype=image_embedding.dtype, device=device)
168:             loss_imgs = loss_img(logits_per_image, ground_truth)
169:             loss_texts = loss_txt(logits_per_text, ground_truth)
170:             total_loss = (loss_imgs + loss_texts) / 2
171:
172:             wandb.log({"train_total_loss": total_loss})
173:             wandb.log({"train_loss_imgs": loss_imgs})
174:             wandb.log({"train_loss_texts": loss_texts})
175:             wandb.log({"lr": optimizer.param_groups[0]['lr']})
176:
177:             total_loss.backward()
178:
179:             if device.type == "cpu":
180:                 optimizer.step()
181:             else:
182:                 convert_models_to_fp32(model)
183:                 optimizer.step()
184:                 clip.model.convert_weights(model)
185:
186:         if epoch % config.logging.eval_freq == 0:
187:             prec1 = validate(epoch, val_loader, classes, device,
model, fusion_model, config, num_text_aug)
188:
189:         is_best = prec1 > best_prec1
190:         best_prec1 = max(prec1, best_prec1)
191:         print(f'Testing: {prec1}/{best_prec1}')
192:         print('Saving:')
193:         filename = f"{working_dir}/last_model.pt"
194:         epoch_saving(epoch, model, fusion_model, optimizer,
filename)
195:         if is_best:
196:             best_saving(working_dir, epoch, model, fusion_model,
optimizer)
```

```
197:
198: if __name__ == '__main__':
199:     main()
```

B.6 test.py (Modified)

```
001: # Code for "ActionCLIP: ActionCLIP: A New Paradigm for Action
Recognition"
002: # arXiv:
003: # Mengmeng Wang, Jiazheng Xing, Yong Liu
004:
005: import os
006: import clip
007: import torch.nn as nn
008: from datasets import Action_DATASETS
009: from torch.utils.data import DataLoader
010: from tqdm import tqdm
011: import wandb
012: import argparse
013: import shutil
014: from pathlib import Path
015: import yaml
016: from dotmap import DotMap
017: import pprint
018: import numpy
019: from modules.Visual_Prompt import visual_prompt
020: from utils.Augmentation import get_augmentation
021: import torch
022: from utils.Text_Prompt import *
023:
024: class TextCLIP(nn.Module):
025:     def __init__(self, model):
026:         super(TextCLIP, self).__init__()
027:         self.model = model
028:
029:     def forward(self, text):
030:         return self.model.encode_text(text)
031:
032: class ImageCLIP(nn.Module):
033:     def __init__(self, model):
034:         super(ImageCLIP, self).__init__()
035:         self.model = model
036:
037:     def forward(self, image):
038:         return self.model.encode_image(image)
039:
040: def validate(epoch, val_loader, classes, device, model,
fusion_model, config, num_text_aug):
041:     from sklearn.metrics import confusion_matrix,
classification_report
042:     import seaborn as sns
043:     import matplotlib.pyplot as plt
044:     import numpy as np
045:
046:     all_preds = []
047:     all_labels = []
048:
049:     model.eval()
050:     fusion_model.eval()
051:     num = 0
052:     corr_1 = 0
```

```
053:     corr_5 = 0
054:
055:     with torch.no_grad():
056:         text_inputs = classes.to(device)
057:         text_features = model.encode_text(text_inputs)
058:         for iii, (image, class_id) in enumerate(tqdm(val_loader)):
059:             image = image.view((-1, config.data.num_segments, 3) +
image.size()[-2:])
060:             b, t, c, h, w = image.size()
061:             class_id = class_id.to(device)
062:             image_input = image.to(device).view(-1, c, h, w)
063:             image_features =
model.encode_image(image_input).view(b, t, -1)
064:             image_features = fusion_model(image_features)
065:             image_features /= image_features.norm(dim=-1,
keepdim=True)
066:             text_features /= text_features.norm(dim=-1,
keepdim=True)
067:             similarity = (100.0 * image_features @
text_features.T)
068:             similarity = similarity.view(b, num_text_aug, -
1).softmax(dim=-1)
069:             similarity = similarity.mean(dim=1, keepdim=False)
070:             preds = similarity.argmax(dim=-1).cpu().numpy()
071:             labels = class_id.cpu().numpy()
072:
073:             all_preds.extend(preds)
074:             all_labels.extend(labels)
075:
076:             values_1, indices_1 = similarity.topk(1, dim=-1)
077:             k_val = min(5, similarity.shape[-1])  #make sure k
does not exceed number of classes
078:             values_5, indices_5 = similarity.topk(k_val, dim=-1)
079:
080:             num += b
081:             for i in range(b):
082:                 if indices_1[i] == class_id[i]:
083:                     corr_1 += 1
084:                 if class_id[i] in indices_5[i]:
085:                     corr_5 += 1
086:     top1 = float(corr_1) / num * 100
087:     top5 = float(corr_5) / num * 100
088:     wandb.log({"top1": top1})
089:     wandb.log({"top5": top5})
090:     print('Epoch: [{}/{}]: Top1: {}, Top5: {}'.format(epoch,
config.solver.epochs, top1, top5))
091:
092:     #confusion Matrix
093:     cm = confusion_matrix(all_labels, all_preds)
094:     plt.figure(figsize=(12, 10))
095:     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
096:     plt.xlabel('Predicted')
097:     plt.ylabel('True')
098:     plt.title('Confusion Matrix')
099:     plt.savefig('confusion_matrix.png')
100:     plt.close()
101:
102:     #classification Report
103:     print("\nClassification Report:")
104:     target_names = ["NormalVideos", "Shoplifting"]
105:     from sklearn.metrics import classification_report
106:
```

```
107:     report = classification_report(all_labels, all_preds,
target_names=target_names)
108:     print(report)
109:
110:     with open("classification_report.txt", "w") as f:
111:         f.write(report)
112:
113:     wandb.log({"confusion_matrix":
wandb.Image('confusion_matrix.png')})
114:
115:     #misclassified samples
116:     print("\nMisclassified Samples:")
117:     for i, (true, pred) in enumerate(zip(all_labels, all_preds)):
118:         if true != pred:
119:             print(f"Sample {i}: True Label: {target_names[true]},
Predicted: {target_names[pred]}")
120:
121:
122:
123:     return top1
124:
125: def main():
126:     global args, best_prec1
127:     global global_step
128:     parser = argparse.ArgumentParser()
129:     parser.add_argument('--config', '-cfg', default='')
130:     parser.add_argument('--log_time', default='')
131:     args = parser.parse_args()
132:
133:     with open(args.config, 'r') as f:
134:         config = yaml.load(f)
135:     working_dir = os.path.join('./exp', config['network']['type'],
config['network']['arch'], config['data']['dataset'],
136:                                args.log_time)
137:     wandb.init(project=config['network']['type'],
138:                name='{}_{}_{}_{}'.format(args.log_time,
config['network']['type'], config['network']['arch'],
139:
config['data']['dataset']))
140:     print('-' * 80)
141:     print(' ' * 20, "working dir: {}".format(working_dir))
142:     print('-' * 80)
143:
144:     print('-' * 80)
145:     print(' ' * 30, "Config")
146:     pp = pprint.PrettyPrinter(indent=4)
147:     pp.pprint(config)
148:     print('-' * 80)
149:
150:     config = DotMap(config)
151:
152:     Path(working_dir).mkdir(parents=True, exist_ok=True)
153:     shutil.copy(args.config, working_dir)
154:     shutil.copy('test.py', working_dir)
155:
156:     device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
157:
158:     model, clip_state_dict = clip.load(config.network.arch,
device=device, jit=False, tsm=config.network.tsm,
159:
T=config.data.num_segments, dropout=config.network.drop_out,
```

```
160:
emb_dropout=config.network.emb_dropout)  #must set jit=False for
training  ViT-B/32
161:
162:     transform_val = get_augmentation(False, config)
163:
164:     fusion_model = visual_prompt(config.network.sim_header,
clip_state_dict, config.data.num_segments)
165:
166:     model_text = TextCLIP(model)
167:     model_image = ImageCLIP(model)
168:
169:     model_text = model_text.to(device)
170:     model_image = model_image.to(device)
171:     fusion_model = fusion_model.to(device)
172:
173:     if device.type != "cpu":
174:         wandb.watch(model)
175:         wandb.watch(fusion_model)
176:
177:     val_data = Action_DATASETS(config.data.val_list,
config.data.label_list, num_segments=config.data.num_segments,
178:                           image_tmpl=config.data.image_tmpl,
179:                           transform=transform_val,
random_shift=config.random_shift)
180:     val_loader = DataLoader(val_data,
batch_size=config.data.batch_size, num_workers=config.data.workers,
shuffle=False,
181:                             pin_memory=True, drop_last=True)
182:
183:     if device == "cpu":
184:         model_text.float()
185:         model_image.float()
186:         fusion_model.float()
187:     else:
188:         clip.model.convert_weights(
189:             model_text)
190:         clip.model.convert_weights(model_image)
191:
192:     start_epoch = config.solver.start_epoch
193:
194:     if config.pretrain:
195:         if os.path.isfile(config.pretrain):
196:             print(("=> loading checkpoint
'{}'".format(config.pretrain)))
197:             checkpoint = torch.load(config.pretrain,
map_location=device)
198:             model.load_state_dict(checkpoint['model_state_dict'])
199:
fusion_model.load_state_dict(checkpoint['fusion_model_state_dict'])
200:             del checkpoint
201:         else:
202:             print(("=> no checkpoint found at
'{}'".format(config.pretrain)))
203:
204:     classes, num_text_aug, text_dict = text_prompt(val_data)
205:
206:     best_prec1 = 0.0
207:     prec1 = validate(start_epoch, val_loader, classes, device,
model, fusion_model, config, num_text_aug)
208:
209: if __name__ == '__main__':
```

```
210:     main()
```

B.7 ucf_train_1shot.yaml

```
01: pretrain:
02: resume:
03: seed: 1024
04: data:
05:     dataset: ucf101
06:     modality: RGB
07:     num_segments: 8
08:     seg_length: 1
09:     split: 1
10:     batch_size: 4
11:     workers: 0
12:     gpus: 2
13:     num_classes: 101
14:     image_tmpl: 'img_{:05d}.jpg'
15:     train_list: 'lists/ucf_few_shot/train_1shot.txt'
16:     val_list: 'lists/ucf101/val_rgb_split1.txt'
17:     label_list: 'lists/ucf_labels.csv'
18:     index_bias: 1
19:     input_size: 224
20:     randaug:
21:         N: 0
22:         M: 0
23: network:
24:     arch: ViT-B/16
25:     init: True
26:     drop_out: 0.0
27:     emb_dropout: 0.0
28:     type: clip_ucf
29:     sim_header: "Transf"  #Transf   meanP   LSTM   Transf_cls
Conv_1D
30:     fix_text: False
31:     fix_img: False
32:     describe:
33: solver:
34:     type: cosine
35:     epochs: 50
36:     start_epoch: 0
37:     epoch_offset: 0
38:     optim: adamw
39:     lr: 5.e-6
40:     lr_warmup_step: 5
41:     momentum: 0.9
42:     weight_decay: 0.2
43:     lr_decay_step: 15
44:     lr_decay_factor: 0.1
45:     clip_gradient: 20
46:     loss_type: nll
47:     evaluate: False
48:     ratio: 1
49:     f_ratio: 10
50: logging:
51:     print_freq: 10
52:     eval_freq: 1
```

B.8 ucf_train_2shot.yaml

```
01: pretrain:  '/public/home/ock/video_clip/video-
clip/clip_models/vit16_8f/model_best.pt'
```

```
02: resume:
03: seed: 1024
04: data:
05:     dataset: ucf101
06:     modality: RGB
07:     num_segments: 8
08:     seg_length: 1
09:     split: 1
10:     batch_size: 4
11:     workers: 0
12:     gpus: 2
13:     num_classes: 101
14:     image_tmpl: 'img_{:05d}.jpg'
15:     train_list: 'lists/ucf_few_shot/train_2shot.txt'
16:     val_list: '/public/home/ock/video_clip/video-
clip/lists/ucf101/val_rgb_split1.txt'
17:     label_list: 'lists/ucf_labels.csv'
18:     index_bias: 1
19:     input_size: 112
20:     randaug:
21:         N: 0
22:         M: 0
23: network:
24:     arch: ViT-B/16
25:     init: True
26:     drop_out: 0.0
27:     emb_dropout: 0.0
28:     type: clip_ucf
29:     sim_header: "Transf"  #Transf   meanP   LSTM   Transf_cls
Conv_1D
30:     fix_text: False
31:     fix_img: False
32:     describe:
33: solver:
34:     type: cosine
35:     epochs: 30
36:     start_epoch: 0
37:     epoch_offset: 0
38:     optim: adamw
39:     lr: 5.e-6
40:     lr_warmup_step: 5
41:     momentum: 0.9
42:     weight_decay: 0.2
43:     lr_decay_step: 15
44:     lr_decay_factor: 0.1
45:     clip_gradient: 20
46:     loss_type: nll
47:     evaluate: False
48:     ratio: 1
49:     f_ratio: 10
50: logging:
51:     print_freq: 10
52:     eval_freq: 1
```

B.9 ucf_train_4shot.yaml

```
01: pretrain:  '/public/home/ock/video_clip/video-
clip/clip_models/vit16_8f/model_best.pt'
02: resume:
03: seed: 1024
04: data:
05:     dataset: ucf101
```

```
    modality: RGB
    num_segments: 8
    seg_length: 1
    split: 1
    batch_size: 1
    workers: 0
    gpus: 2
    num_classes: 101
    image_tmpl: 'img_{:05d}.jpg'
    train_list: 'lists/ucf_few_shot/train_4shot.txt'
    val_list: '/public/home/ock/video_clip/video-
clip/lists/ucf101/val_rgb_split1.txt'
    label_list: 'lists/ucf_labels.csv'
    index_bias: 1
    input_size: 112
    randaug:
        N: 0
        M: 0
network:
    arch: ViT-B/16  #ViT-B/32 ViT-B/16
    init: True
    drop_out: 0.0
    emb_dropout: 0.0
    type: clip_ucf
    sim_header: "Transf"  #Transf   meanP   LSTM   Transf_cls
Conv_1D
    fix_text: False
    fix_img: False
    describe:
solver:
    type: cosine
    epochs: 10
    start_epoch: 0
    epoch_offset: 0
    optim: adamw
    lr: 5.e-6
    lr_warmup_step: 5
    momentum: 0.9
    weight_decay: 0.2
    lr_decay_step: 15
    lr_decay_factor: 0.1
    clip_gradient: 20
    loss_type: nll
    evaluate: False
    ratio: 1
    f_ratio: 10
logging:
    print_freq: 10
    eval_freq: 1
```

B.10 ucf_crime_train.yaml

```
network:
  type: clip_ucf
  arch: ViT-B/16
  tsm: False
  sim_header: Transf
  drop_out: 0.0
  emb_dropout: 0.0
  fix_text: False
  init: ''
  joint: False
```

```
11:
12: data:
13:   dataset: ucf_crime
14:   modality: RGB
15:   train_list: lists/ucf_crime/train_rgb.txt
16:   val_list: lists/ucf_crime/val_rgb.txt
17:   label_list: lists/ucf_crime/crime_labels.csv
18:   num_segments: 8
19:   image_tmpl: 'img_{:05d}.jpg'
20:   prefix: ''
21:   workers: 4
22:   batch_size: 4
23:   random_shift: True
24:   train_length: 1
25:   val_length: 1
26:   input_size: 224
27:   randaug:
28:     N: 0
29:     M: 0
30:
31: solver:
32:   type: cosine
33:   optim: adamw
34:   lr: 1e-5
35:   ratio: 1.0
36:   epochs: 10
37:   start_epoch: 0
38:   warmup_epochs: 2.5
39:   base_lr: 1e-5
40:   lr_mult: 5
41:   weight_decay: 0.2
42:   evaluate: False
43:   resume: ''
44:   print_freq: 10
45:
46: logging:
47:   eval_freq: 1
48:
49: pretrain: exp/clip_ucf/ViT-
B/16/ucf101/20250415_204253/model_best.pt
```

B.11 prompt_ensemble.py

```
01: import torch
02: from clip import tokenize
03: import numpy as np
04:
05: def generate_prompt_ensemble_embeddings(class_prompt_dict,
clip_model, device):
06:     """
07:     Generate averaged text embeddings for each class using multiple
prompt variations.
08:
09:     Args:
10:         class_prompt_dict (dict): {class_name: [prompt1, prompt2,
...]}
11:         clip_model (nn.Module): Loaded CLIP model
12:         device (torch.device): CPU or CUDA
13:
14:     Returns:
15:         torch.Tensor: Averaged text embeddings of shape
(num_classes, embed_dim)
```

```
16:     """
17:     class_embeddings = []
18:
19:     for class_name, prompts in class_prompt_dict.items():
20:         tokens = tokenize(prompts).to(device)
21:
22:         with torch.no_grad():
23:             embeddings = clip_model.encode_text(tokens)
24:             embeddings = embeddings / embeddings.norm(dim=-1,
keepdim=True)
25:
26:         mean_embedding = embeddings.mean(dim=0)
27:         mean_embedding = mean_embedding / mean_embedding.norm()
28:         class_embeddings.append(mean_embedding)
29:
30:     return torch.stack(class_embeddings, dim=0)
```

B.12 anomaly_test.py

```
001:
002: import os
003: import torch
004: from torch.utils.data import Subset
005: import torch.nn as nn
006: import numpy as np
007: from torch.utils.data import DataLoader
008: from sklearn.metrics import accuracy_score, precision_score,
recall_score, roc_auc_score, confusion_matrix
009: from tqdm import tqdm
010: from datasets import Action_DATASETS
011: from utils.Augmentation import get_augmentation
012: from modules.Visual_Prompt import visual_prompt
013: import clip
014: import yaml
015: from dotmap import DotMap
016: import matplotlib.pyplot as plt
017: import seaborn as sns
018:
019:
020: class ImageCLIP(nn.Module):
021:     def __init__(self, model):
022:         super(ImageCLIP, self).__init__()
023:         self.model = model
024:
025:     def forward(self, image):
026:         return self.model.encode_image(image)
027:
028: def compute_centroid(embeddings):
029:     return torch.stack(embeddings).mean(dim=0)
030:
031: def cosine_distance(a, b):
032:     a_norm = a / a.norm(dim=-1, keepdim=True)
033:     b_norm = b / b.norm(dim=-1, keepdim=True)
034:     return 1 - torch.sum(a_norm * b_norm, dim=-1)
035:
036: def extract_embeddings(model, fusion_model, dataloader, device,
max_batches):
037:     model.eval()
038:     fusion_model.eval()
039:     embeddings = []
040:     labels = []
041:     with torch.no_grad():
```

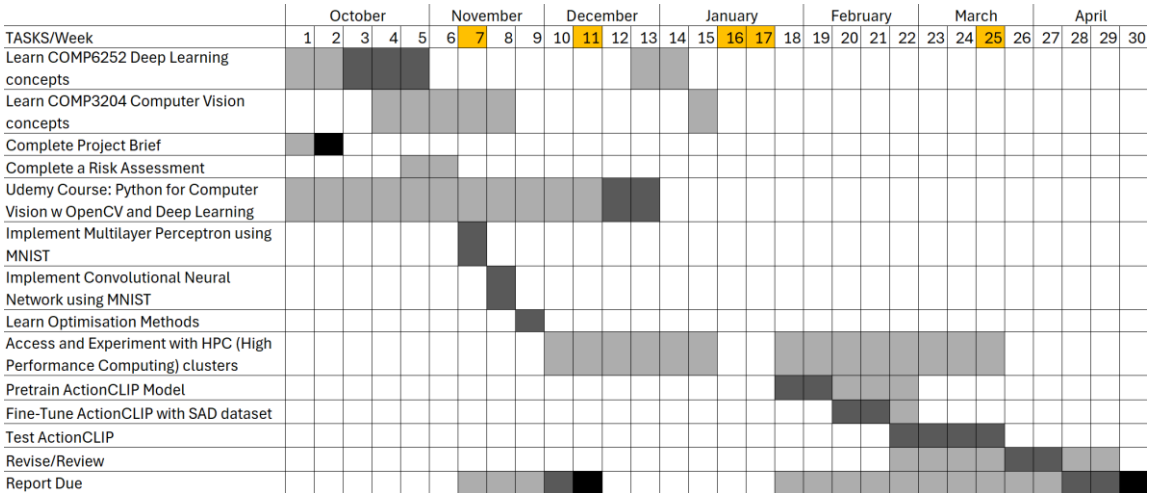
```
042:         for i, (images, targets) in enumerate(tqdm(dataloader)):
043:             if max_batches is not None and i >= max_batches:
044:                 break
045:             images = images.view((-1, config.data.num_segments, 3)
+ images.size()[-2:])
046:             b, t, c, h, w = images.size()
047:             images = images.to(device).view(-1, c, h, w)
048:             targets = targets.to(device)
049:
050:             image_features = model(images)
051:             image_features = image_features.view(b, t, -1)
052:             image_features = fusion_model(image_features)
053:             embeddings.extend(image_features)
054:             labels.extend(targets)
055:     return embeddings, labels
056:
057:
058: def evaluate_anomalies(distances, true_labels, threshold=None):
059:     scores = np.array(distances)
060:     labels = np.array(true_labels)
061:
062:     if threshold is None:
063:         threshold = np.percentile(scores[labels == 0], 95)
064:
065:     preds = (scores > threshold).astype(int)
066:     acc = accuracy_score(labels, preds)
067:     prec = precision_score(labels, preds)
068:     rec = recall_score(labels, preds)
069:     auc = roc_auc_score(labels, scores)
070:     cm = confusion_matrix(labels, preds)
071:
072:     print(f"Threshold: {threshold:.4f}")
073:     print(f"Accuracy: {acc:.4f}, Precision: {prec:.4f}, Recall:
{rec:.4f}, AUC: {auc:.4f}")
074:     print("Confusion Matrix:")
075:     print(cm)
076:
077:     plt.figure(figsize=(6, 5))
078:     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
079:     plt.xlabel("Predicted")
080:     plt.ylabel("True")
081:     plt.title("Confusion Matrix")
082:     plt.savefig("anomaly_confusion_matrix.png")
083:     plt.close()
084:
085:     from sklearn.metrics import roc_curve
086:     fpr, tpr, _ = roc_curve(labels, scores)
087:     plt.figure(figsize=(6, 5))
088:     plt.plot(fpr, tpr, label=f'AUC = {auc:.4f}')
089:     plt.plot([0, 1], [0, 1], 'k--')
090:     plt.xlabel('False Positive Rate')
091:     plt.ylabel('True Positive Rate')
092:     plt.title('ROC Curve')
093:     plt.legend(loc='lower right')
094:     plt.savefig("roc_curve.png")
095:     plt.close()
096:
097:     with open("anomaly_metrics_report.txt", "w") as f:
098:         f.write(f"Threshold: {threshold:.4f}\n")
099:         f.write(f"Accuracy: {acc:.4f}\n")
100:         f.write(f"Precision: {prec:.4f}\n")
101:         f.write(f"Recall: {rec:.4f}\n")
```

```
102:         f.write(f"AUC: {auc:.4f}\n")
103:         f.write("Confusion Matrix:\n")
104:         f.write(str(cm))
105:
106:
107: if __name__ == '__main__':
108:     CONFIG_PATH = 'configs/ucf_crime/ucf_crime_train.yaml'
109:     CHECKPOINT_PATH = 'exp/clip_ucf/ViT-
B/16/ucf101/20250403_230130/model_best.pt'
110:
111:     with open(CONFIG_PATH, 'r') as f:
112:         config = DotMap(yaml.safe_load(f))
113:
114:     device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
115:
116:
117:     clip_model, clip_state_dict = clip.load(config.network.arch,
device=device, jit=False,
118:
tsm=config.network.tsm, T=config.data.num_segments,
119:
dropout=config.network.drop_out,
120:
emb_dropout=config.network.emb_dropout)
121:     fusion_model = visual_prompt(config.network.sim_header,
clip_state_dict, config.data.num_segments)
122:
123:     model_image = ImageCLIP(clip_model).to(device)
124:     fusion_model = fusion_model.to(device)
125:
126:     if os.path.isfile(CHECKPOINT_PATH):
127:         checkpoint = torch.load(CHECKPOINT_PATH,
map_location=device)
128:         clip_model.load_state_dict(checkpoint['model_state_dict'],
strict=False)
129:
130:     transform = get_augmentation(False, config)
131:
132:     train_set = Action_DATASETS(config.data.train_list,
config.data.label_list,
133:
num_segments=config.data.num_segments,
134:                                 image_tmpl=config.data.image_tmpl,
135:                                 transform=transform,
random_shift=False)
136:     #train_set = Subset(train_set, list(range(10)))
137:     test_set = Action_DATASETS(config.data.val_list,
config.data.label_list,
138:
num_segments=config.data.num_segments,
139:                                image_tmpl=config.data.image_tmpl,
140:                                transform=transform,
random_shift=False)
141:     #test_set = Subset(test_set, list(range(10)))
142:
143:
144:     train_loader = DataLoader(train_set,
batch_size=config.data.batch_size, shuffle=False, num_workers=4)
145:     test_loader = DataLoader(test_set,
batch_size=config.data.batch_size, shuffle=False, num_workers=4)
146:
```

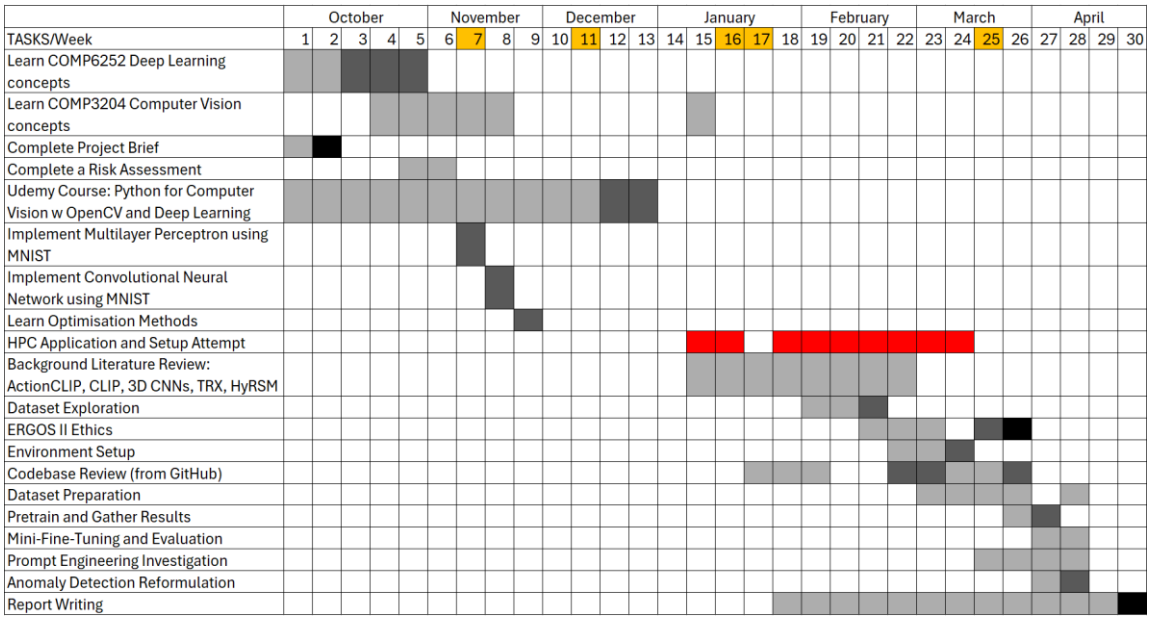
```
147:
148:     print("Extracting NormalVideos embeddings (centroid)...")
149:     train_embeddings, _ = extract_embeddings(model_image,
fusion_model, train_loader, device, max_batches=2500)
150:     normal_centroid = compute_centroid(train_embeddings)
151:
152:     print("Evaluating on test set...")
153:     test_embeddings, test_labels = extract_embeddings(model_image,
fusion_model, test_loader, device, max_batches=2500)
154:     distances = [cosine_distance(embed, normal_centroid).item()
for embed in test_embeddings]
155:
156:     evaluate_anomalies(distances, test_labels)
```

Appendix C. Project Planning and Risk Assessment

C.1 Proposed Project Timeline: Gantt Chart of Initial Plan



C.2 Actual Project Timeline: Gantt Chart of Final Execution



Orange week boxes - represent the weeks when I have other assignment deadlines/exams
Light grey boxes – represent the weeks when I could multitask
Dark grey boxes – represent the weeks where I must prioritise a task
Black boxes – represent the weeks where a submission ***is due / was made***
Red boxes – represent tasks that were eventually changed / not needed

C.3 Risk Assessment Form

Category	Risk	Likelihood	Impact	Mitigation
Technical	Inability to access GPU on HPC (Iridis 5)	Medium	High	Convert ActionCLIP for CPU; reduce number of segments and batch size
	Prolonged training time on CPU	High	High	Reframe problem as anomaly detection; minimise fine-tuning cycles
	Software dependency conflicts	Medium	Medium	Lock versions via environment.yml ; test environment locally
	Incompatibility with CLIP's cuda functions	Medium	Low	Audit and modify GPU calls in codebase for CPU fallback
	Inadequate model convergence due to short training durations	Medium	Medium	Use pretrained embeddings; rely on statistical scoring methods instead
Legal	Dataset licensing conflicts	Medium	High	Switch to CC0-licensed datasets
	Reuse of third-party code without attribution	Low	Medium	Provide proper citation to ActionCLIP repo and preprints
Ethical	Use of real surveillance footage without ethical clearance	Medium	High	Submit ERGO II application; restrict to approved public datasets
	Potential presence of PII in video datasets	Low	High	Use anonymised, low-resolution datasets; avoid facial identification
Project Management	Delay in pivot after HPC denial	High	High	Reallocate time to dataset preparation and CPU adaptation
	Overrun during report writing phase	Low	Medium	Begin early drafting in March, log diagrams and metrics incrementally
	Underestimation of time needed for further experimentation	Medium	High	Limit to experimentation on visual and textual encoders only
Operational	Local hardware unable to process full dataset	High	Medium	Subsample data; process offline in batches
	Inconsistent training/evaluation behaviour across runs	Low	Medium	Log all config values; run multiple trials where possible
	Storage limitations due to extracted frame volume	High	Low	Delete raw video after extraction; compress old logs and unused files
Compliance and Regulatory	Breach of data usage terms from external repositories	Low	High	Double-check dataset licenses (Kaggle, UCF); cite clearly in report
	Failure to comply with research ethics procedure	Low	High	Include risk assessments, dataset documentation, and ethics approval in report

	High energy consumption from extended training runs	Medium	Medium	Optimise batch sizes, reduce model retraining; run on CPU where feasible
Environmental	Impact of lighting and weather conditions on video samples from dataset	Low	Low	Use preprocessing techniques to enhance video quality; train model on varied environmental data

C.4 Project Brief

1. Problem

Action recognition refers to the task of identifying actions in videos, typically using machine learning models. It has evolved through feature engineering to architecture engineering, where modern deep learning models are used to recognise actions from video inputs. These models are typically trained to classify a fixed set of predefined actions, which works well for known, labelled datasets but struggles to generalise to new actions or domains. This is where the concept of zero-shot action recognition becomes crucial. Traditional models require extensive retraining when faced with unfamiliar actions, which limits their flexibility. Zero-shot action recognition, on the other hand, allows models to recognise unseen actions by leveraging semantic information from natural language descriptions or textual labels. However, models that don't incorporate this capability are severely limited, as they require new labelled data for every unseen action, making their deployment in dynamic environments costly in terms of training time and data.

In security surveillance, action recognition models are employed to monitor live video feeds, often with the goal of detecting irregular or suspicious activities. The challenge here is that most conventional models are trained on general action datasets, which may not be representative of the specific types of actions relevant in surveillance, such as theft, loitering, or trespassing. As a result, they often struggle with accuracy when applied to real-time scenarios that deviate from the data they were trained on. Moreover, since security environments are highly dynamic, retraining models to identify emerging behaviours or new types of suspicious activities is a constant necessity. This makes it impractical to rely solely on traditional action recognition methods. The lack of effective zero-shot recognition in this field further raises the issue, limiting a system's ability to adapt to new threats without requiring large volumes of new training data.

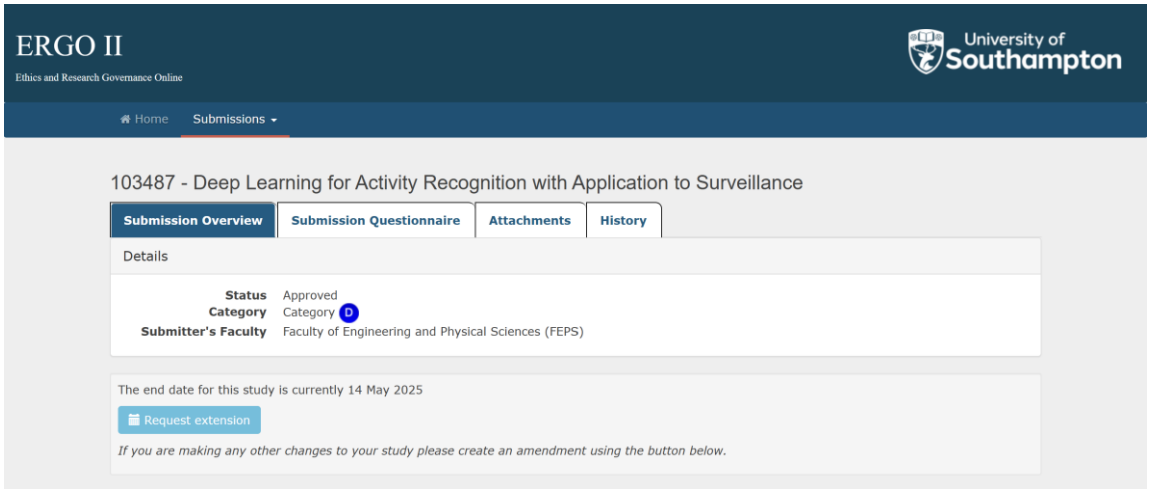
2. Goals

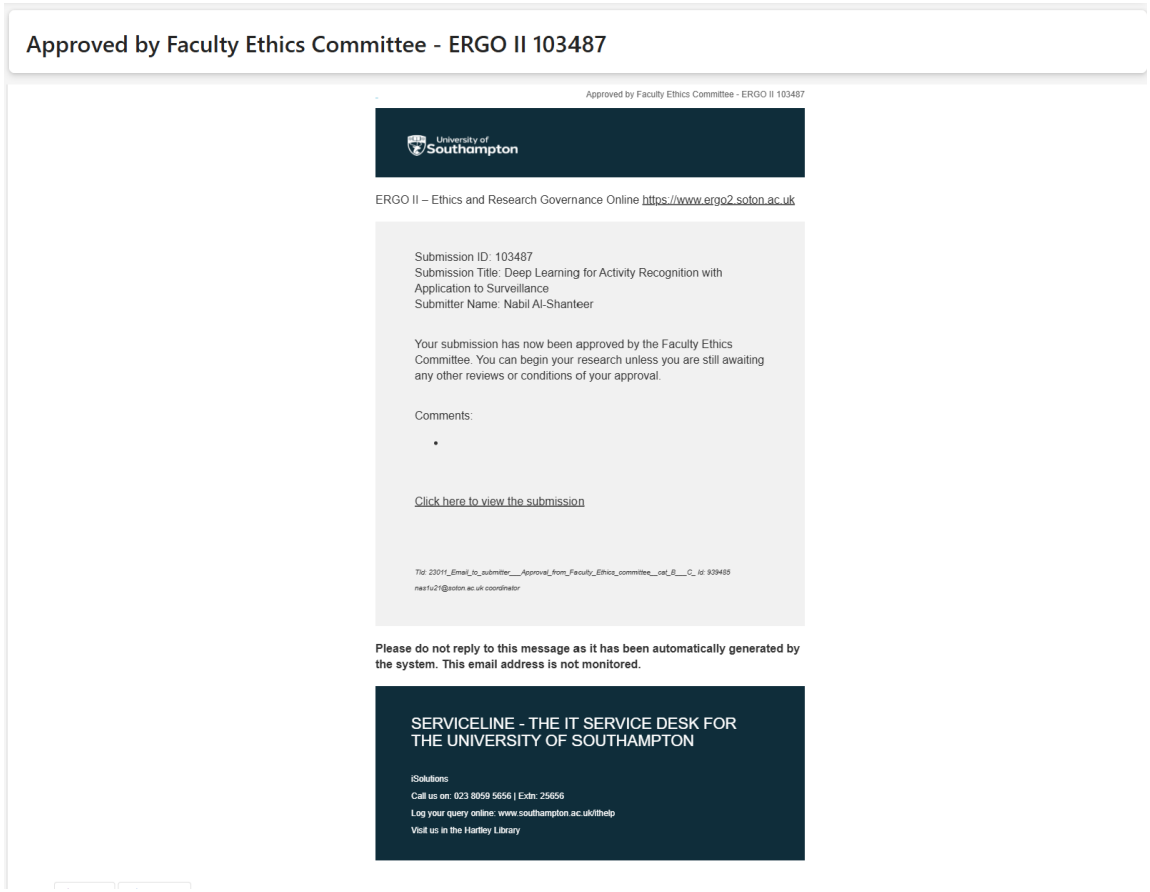
The primary goal of this project is to reproduce and extend a deep learning-based action recognition model, following the methodologies outlined in the research paper, "ActionCLIP: A New Paradigm for Video Action Recognition". The first objective is to replicate the model's architecture and functionality as presented in the paper, ensuring that the baseline model performs effectively on publicly available datasets via GitHub or other community repositories. This baseline will then be fine-tuned and adapted for the surveillance domain. The next step is to train the model with publicly available video data that emphasises irregular or suspicious activities, such as theft and loitering, to provide added relevance for security surveillance applications security applications. Given that deep learning models are data hungry, another goal is to gather and integrate extensive, domain-specific datasets to improve the model's accuracy in real-time security contexts. In addition, ensuring that the model can generalise to unseen actions through zero-shot or few-shot learning techniques is critical, as this will allow the system to detect new and uncommon activities without requiring extensive retraining.

3. Scope

Reproducing the deep learning-based action recognition model using public datasets, with a focus on achieving high accuracy on a general dataset (like Kinetics or UCF101). Inspired by the referenced paper, "ActionCLIP", the project will follow a pre-train, prompt, and fine-tune strategy for the deep learning model, allowing for increased adaptability across various surveillance scenarios, with a particular emphasis on suspicious activities that are potentially done with criminal intent. In terms of testing the model, it will be implemented in a real-time environment using live camera feeds to test its performance in recognizing actions as they happen. This raises a question about ethics regarding data privacy, which will be a topic of high priority to be tackled early on in the project to ensure all testing is legal and preserves the rights of the individuals involved. Finally, the generalisation of zero-shot or few-shot learning techniques will be incorporated to maintain the model's functionality in unseen dynamic environments.

Appendix D. ERGO II Secondary Data Analysis Ethics Approval





Appendix E. Archive Folder Contents

The archive contains the codebase from Reference [92] with some modifications – which are all detailed in Appendix B

ActionCLIP
- clip
  - __init__.py
  - bpe_simple_vocab_16e6.txt.gz
  - clip.py
  - model_zoo.py
  - model.py
  - simple_tokenizer.py
- configs
  - hmdb51
  - k400
  - ucf_crime
    - ucf_crime_train.yaml
  - ucf101
    - ucf_test.yaml
    - ucf_train_1shot.yaml
    - ucf_train_2shot.yaml
    - ucf_train_4shot.yaml
    - ucf_train.yaml
    - ucf_zero_shot.yaml
- datasets
  - __init__.py
  - datasets.py
  - transforms_ss.py
- exp
- lists
  - k4001
  - ucf_crime
    - crime_labels.csv
    - train_rgb.txt
    - val_rgb.txt
  - ucf_few_shot
    - train_1shot.txt
    - train_2shot.txt
    - train_4shot.txt
  - ucf101
    - val_rgb_split1.txt
  - generate_few_shot_splits.py
  - ucf_labels.csv
- logs
- modules
  - basic_ops.py
  - temporal_shift.py
  - Text_Prompt.py
  - Visual_Prompt.py
- scripts
  - extract_frames.py
  - run_test.sh
  - run_train.sh
- utils
  - Augmentation.py
  - KLLoss.py
  - lr_scheduler.py
  - saving.py

solver.py
Text_Prompt.py
tools.py
wandb
.gitignore
ActionCLIP.jpg
ActionCLIP.png
anomaly_confusion_matrix.png
anomaly_test.py
environment.yml
frame_extract.py
generate_crime.py
INSTALL.md
LICENSE
MODEL_ZOO.md
prompt_ensemble.py
README.md
setup.py
test.py
train.py
validation_split.py